

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 3: GRAPH TRAVERSAL ALGORITHMS DFS-BFS

Abdullah Bal, PhD

Objectives



Recap

Exhaustive search

Travelling salesman problem

Knapsack problem

Assignment problem



Graph Traversal



Depth First Search (DFS)

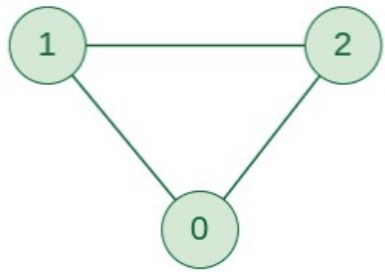


Breadth First Search (BFS)

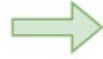
Graph Traversal Algorithms

- Many problems require processing all graph vertices (and edges) in systematic fashion
- Graph traversal algorithms:
 - Depth-first search (DFS)
 - Breadth-first search (BFS)

Graph Representation

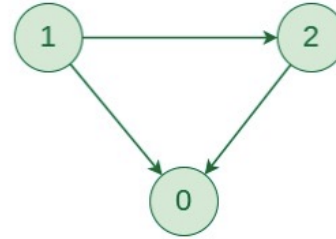


Undirected Graph



	0	1	2
0		1	1
1	1		1
2	1	1	

Adjacency Matrix

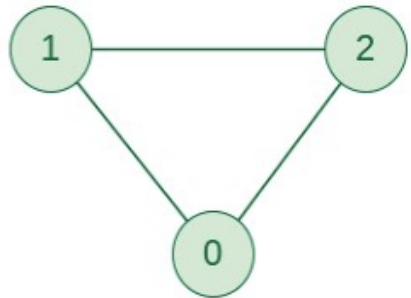


Directed Graph

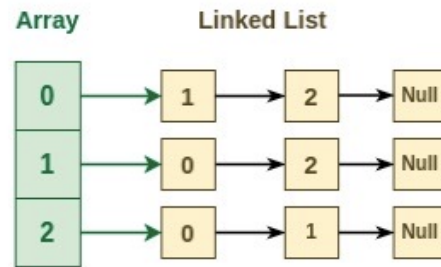
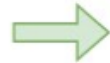


	0	1	2
0			
1	1		1
2	1		

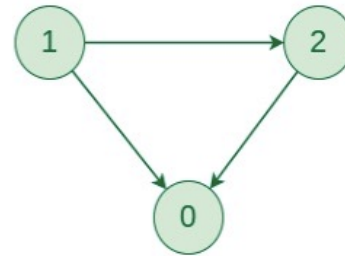
Adjacency Matrix



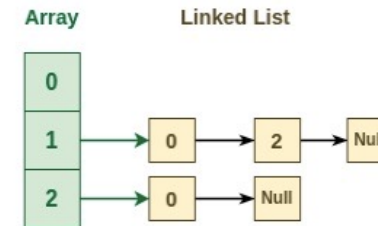
Undirected Graph



Adjacency List



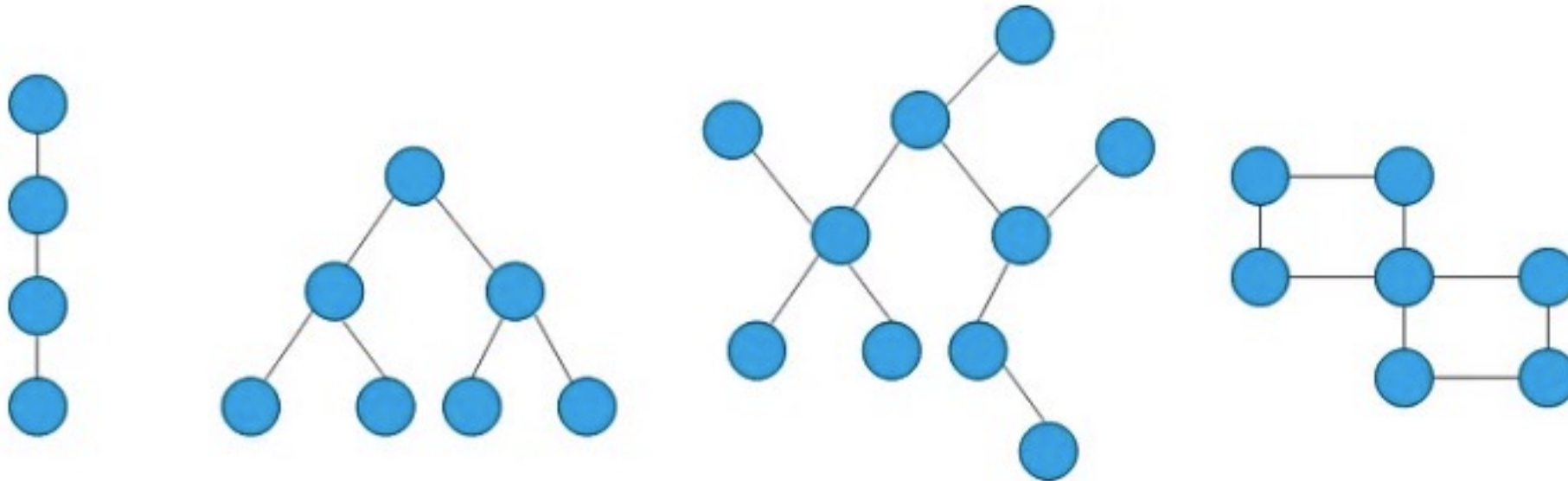
Directed Graph



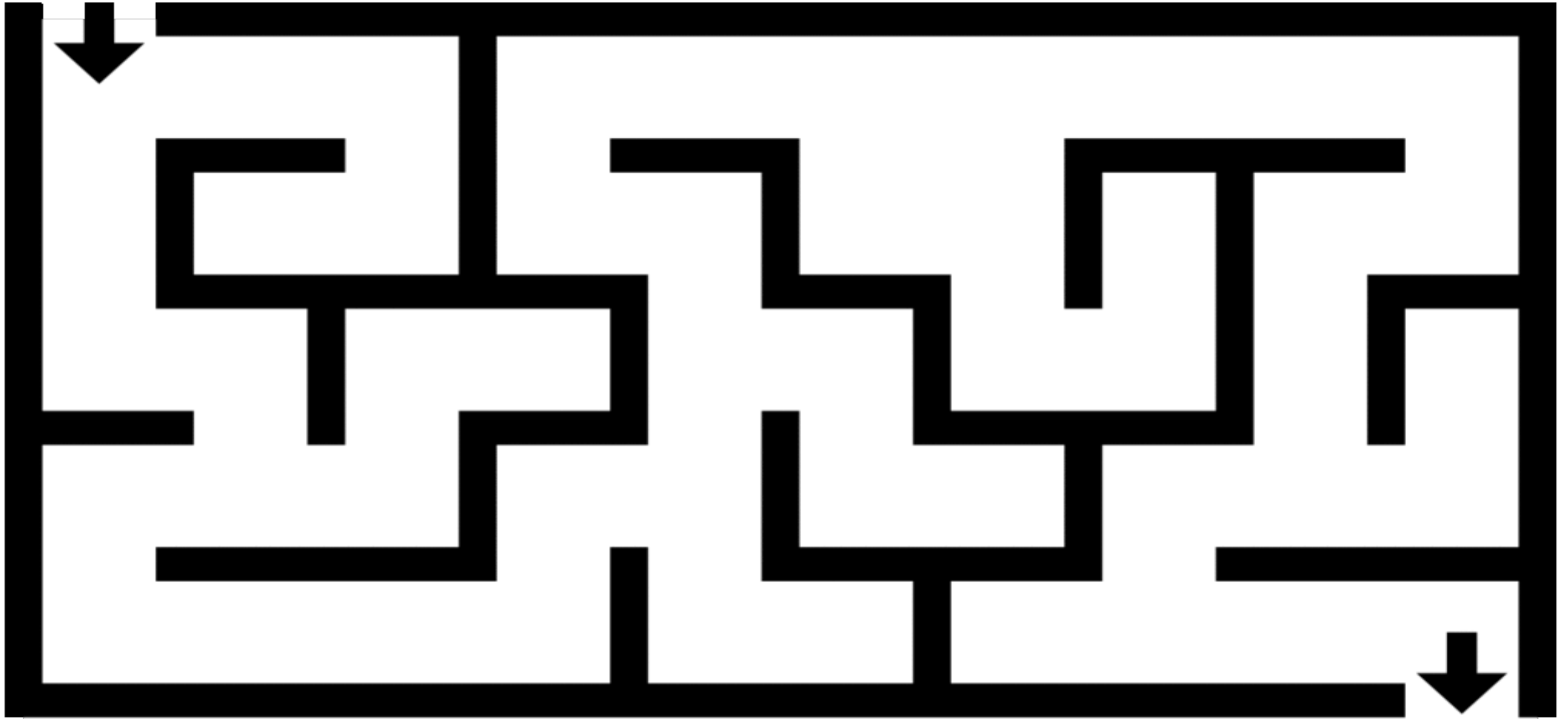
Adjacency List

Tree

- Tree is an **undirected graph** with **no cycles**.
- All trees have **$N - 1$ edges**, where N is the number of nodes.



Depth-First Search (DFS)



Depth-First Search (DFS)

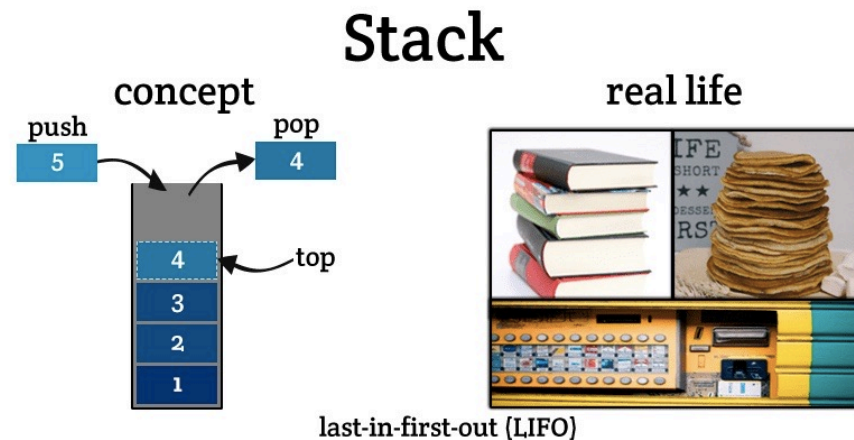
- Depth-first search starts a graph's traversal at **an arbitrary vertex** by marking it as visited.
- On each iteration, the algorithm **proceeds to an unvisited vertex** that is adjacent to the one it is currently in.
- If there are several such vertices, a tie can be **resolved arbitrarily**.
- As a practical matter, which of the adjacent unvisited candidates is chosen is dictated by the data structure representing the graph.
- In our examples, we always break ties by **the alphabetical order of the vertices**.

Depth-First Search (DFS)

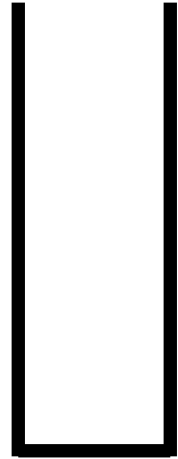
- This process continues **until a dead end**—a vertex with no adjacent unvisited vertices— is encountered.
- At a dead end, the algorithm backs up one edge to the vertex it came from and **tries to continue visiting unvisited vertices from there**.
- The algorithm **eventually halts after backing up to the starting vertex**, with the latter being a dead end.
- By then, all the vertices in the same connected component as the starting vertex have been visited.
- If unvisited vertices still remain, the depth-first search must be **restarted at any one of them**.

Depth-First Search (DFS)

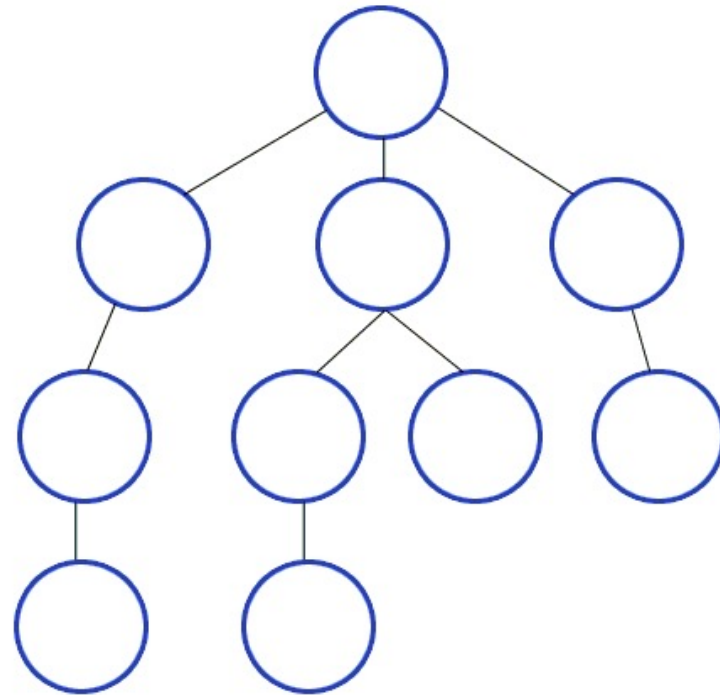
- Visits graph's vertices by always moving away from last visited vertex to unvisited one, backtracks if no adjacent unvisited vertex is available.
- Uses a **stack**
 - a vertex is pushed onto the stack when it's reached for the first time
 - a vertex is popped off the stack when it becomes a dead end, i.e., when there is no adjacent unvisited vertex



Depth-First Search (DFS)

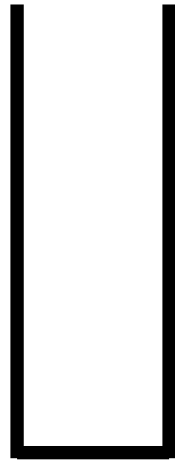


Stack

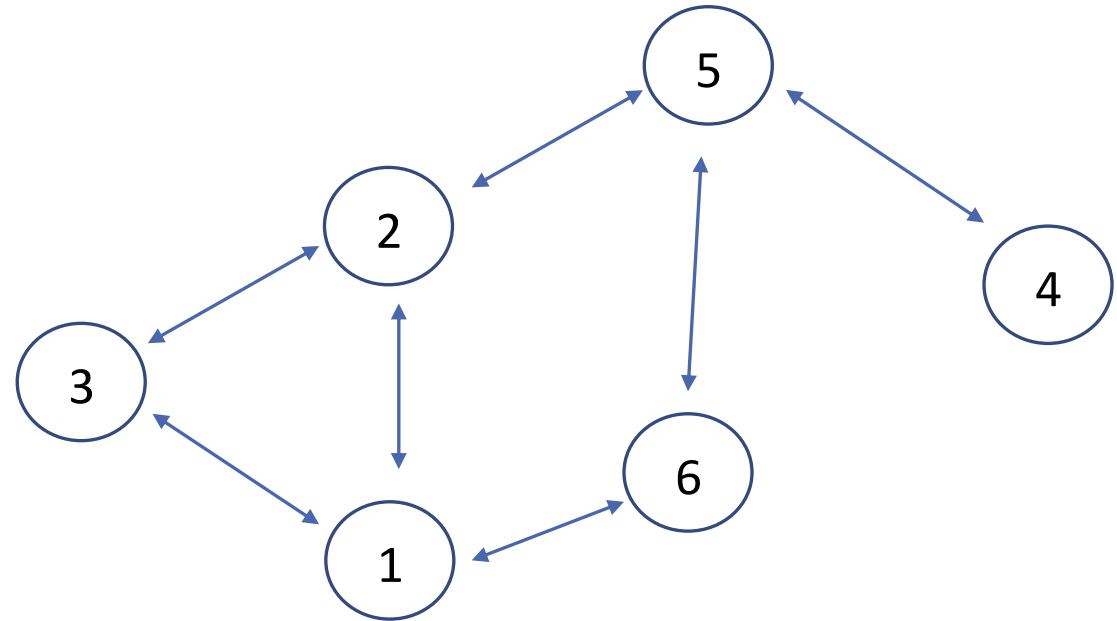


Example 1: Depth-First Search (DFS)

- The essence of DFS: “Go into deep”
- Go as far from “home” as it can

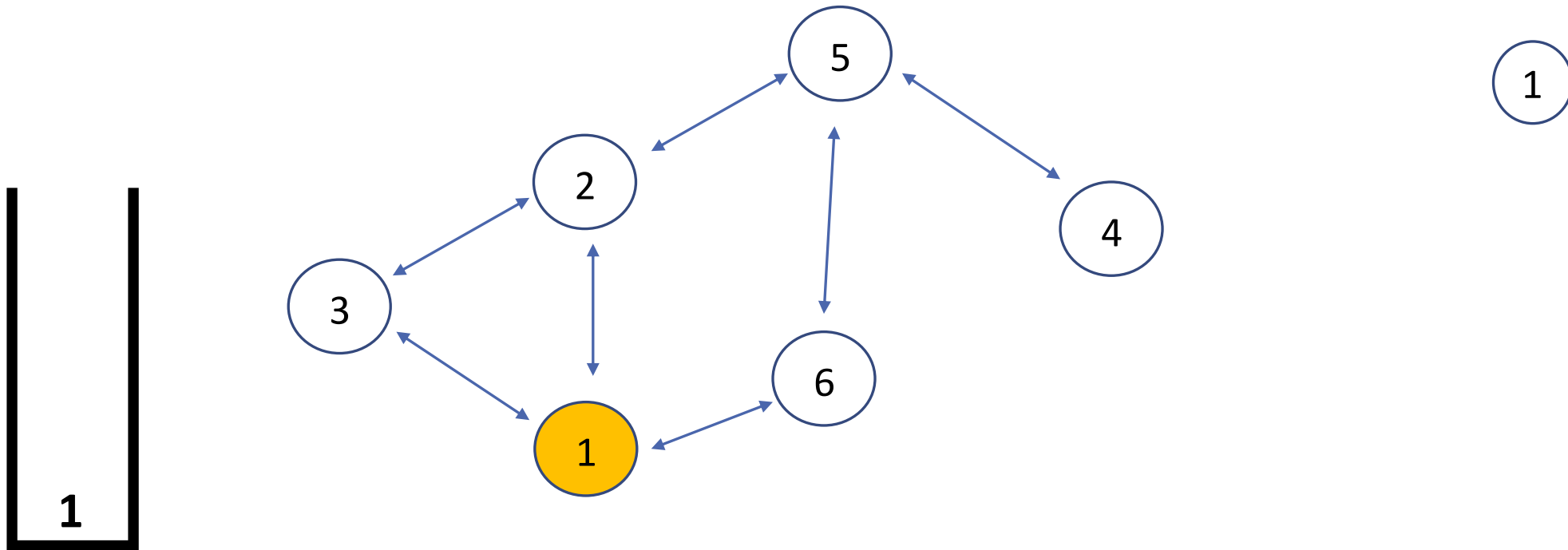


Stack



DFS Result:

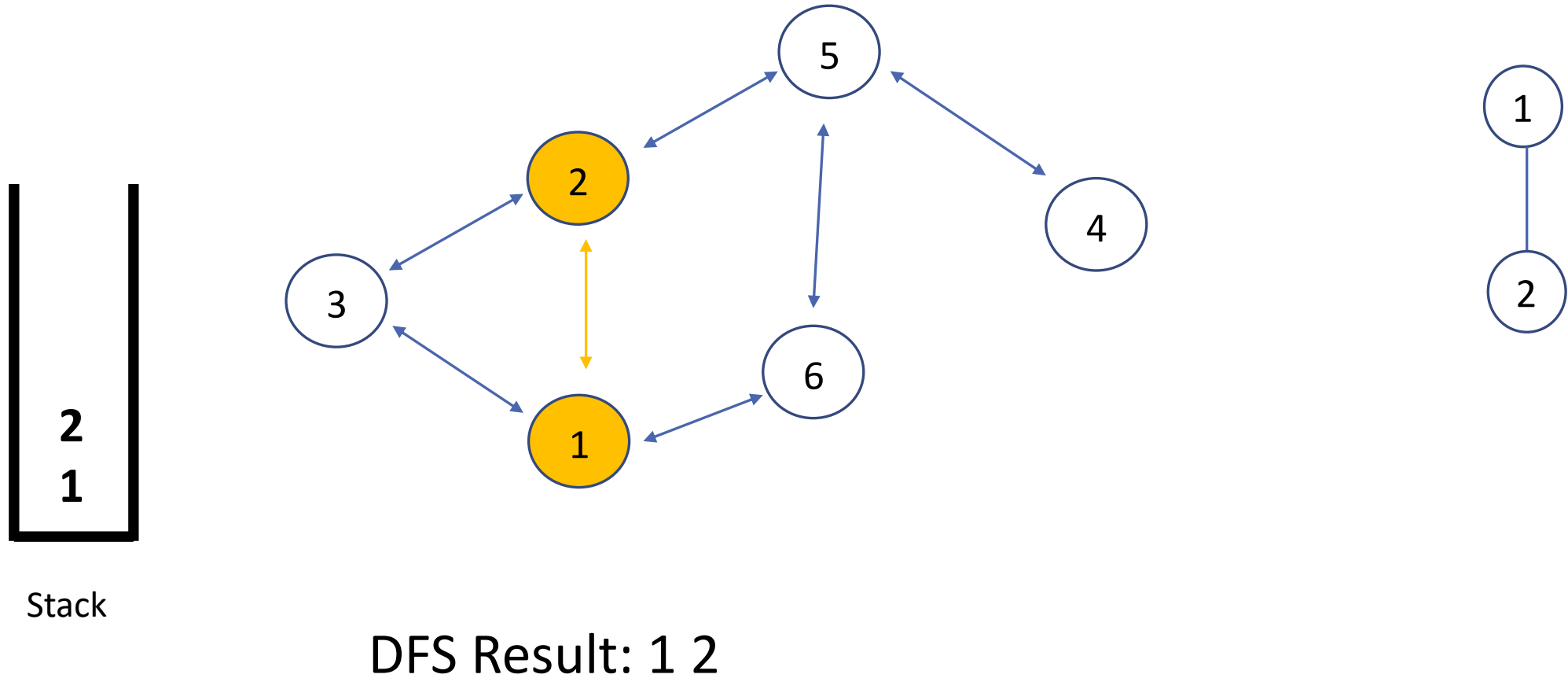
Example 1: Depth-First Search (DFS)



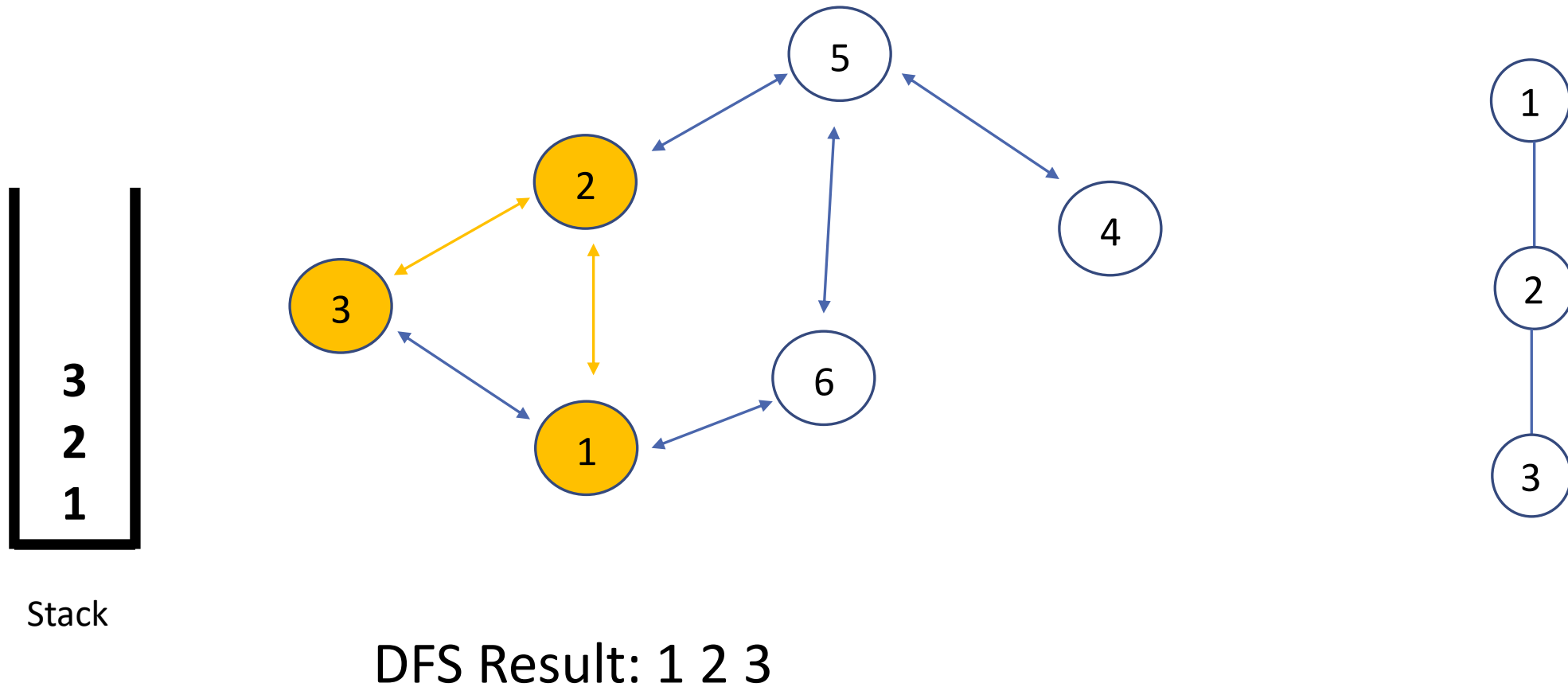
Stack

DFS Result: 1

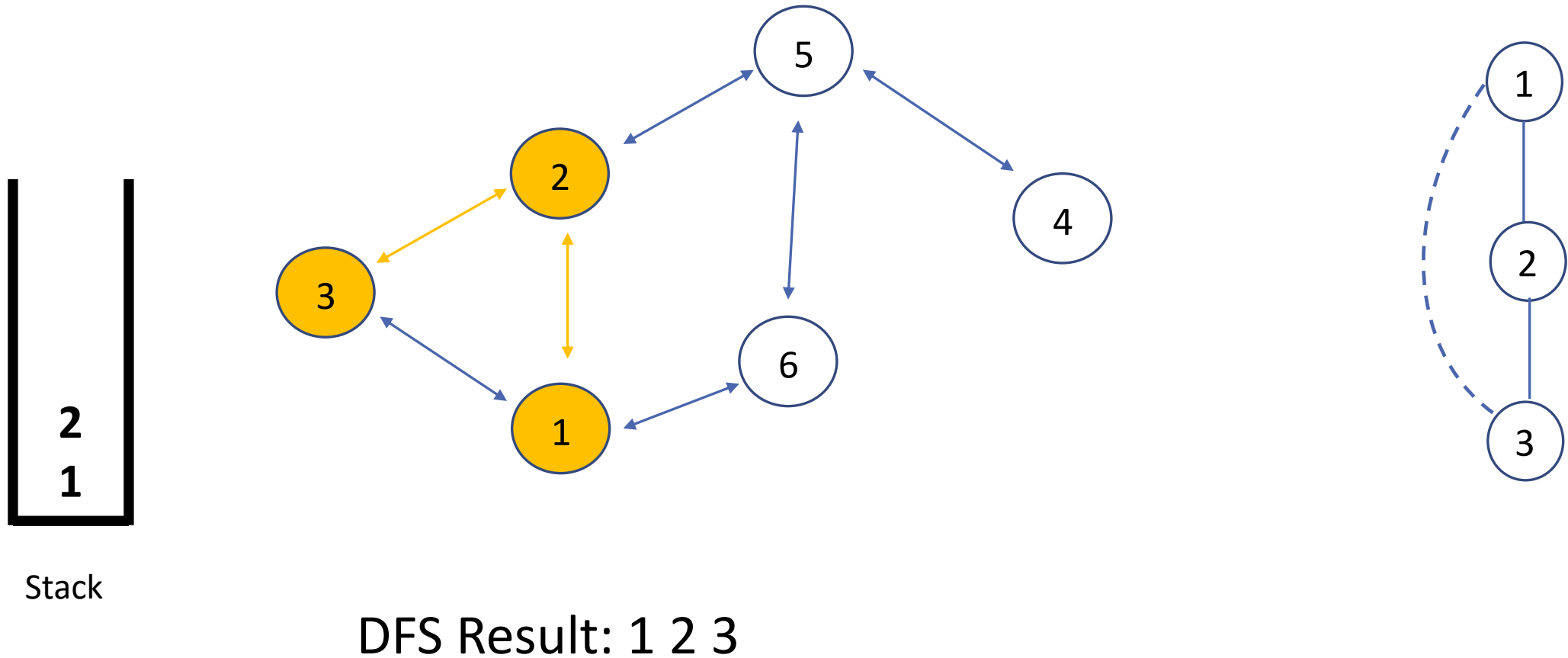
Example 1: Depth-First Search (DFS)



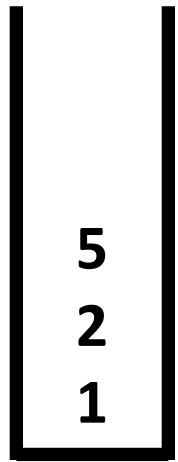
Example 1: Depth-First Search (DFS)



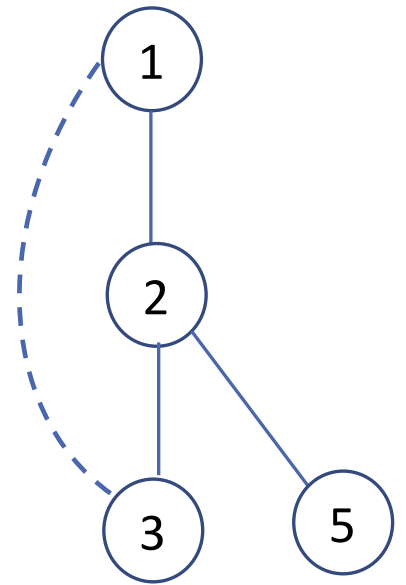
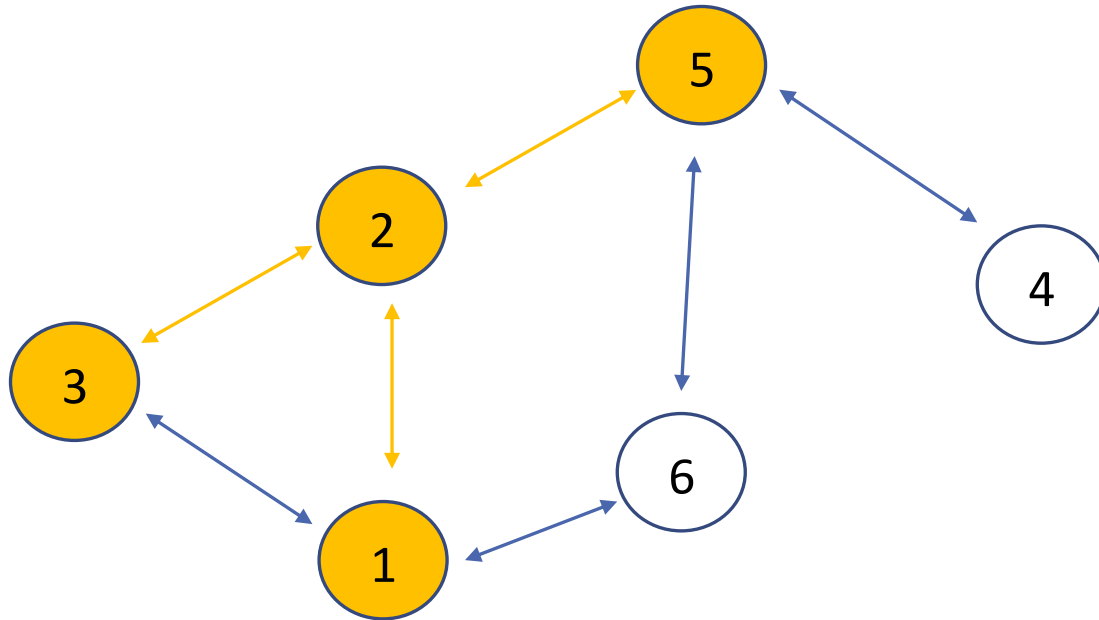
Example 1: Depth-First Search (DFS)



Example 1: Depth-First Search (DFS)

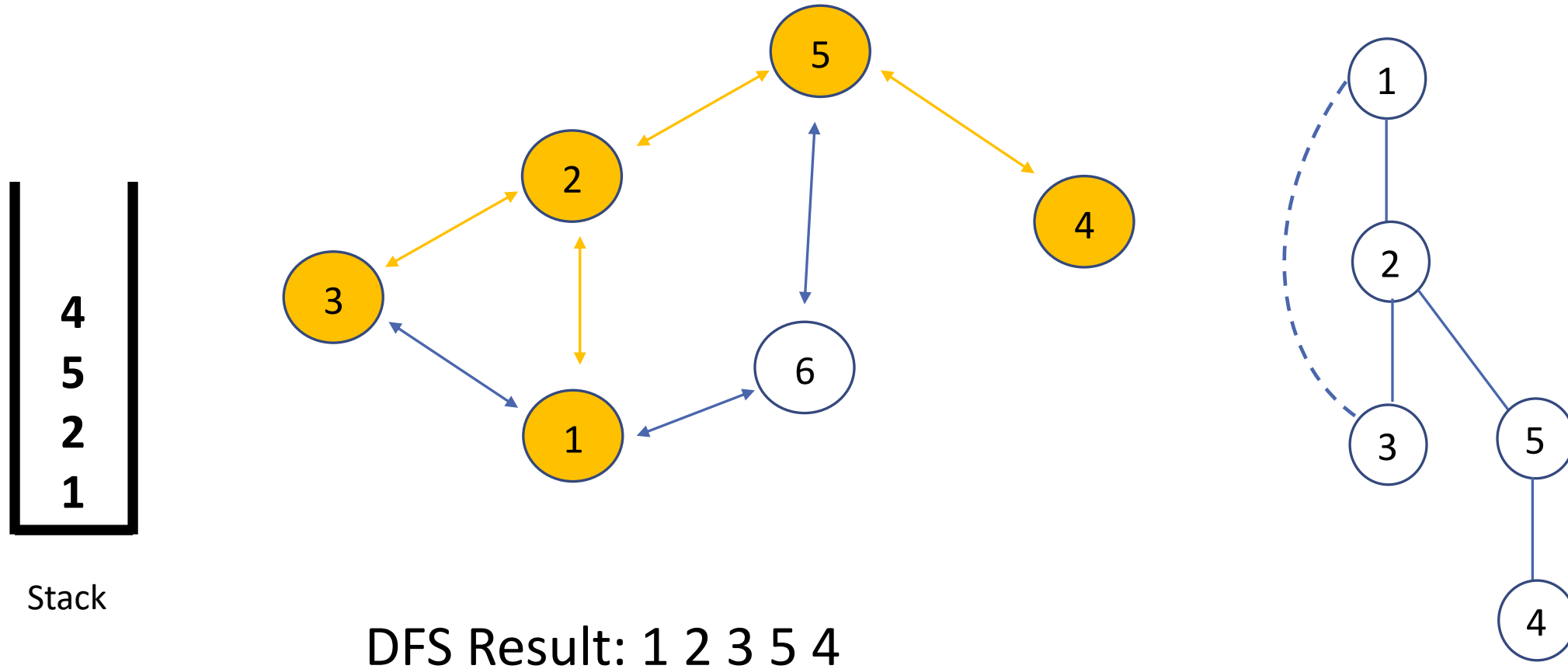


Stack

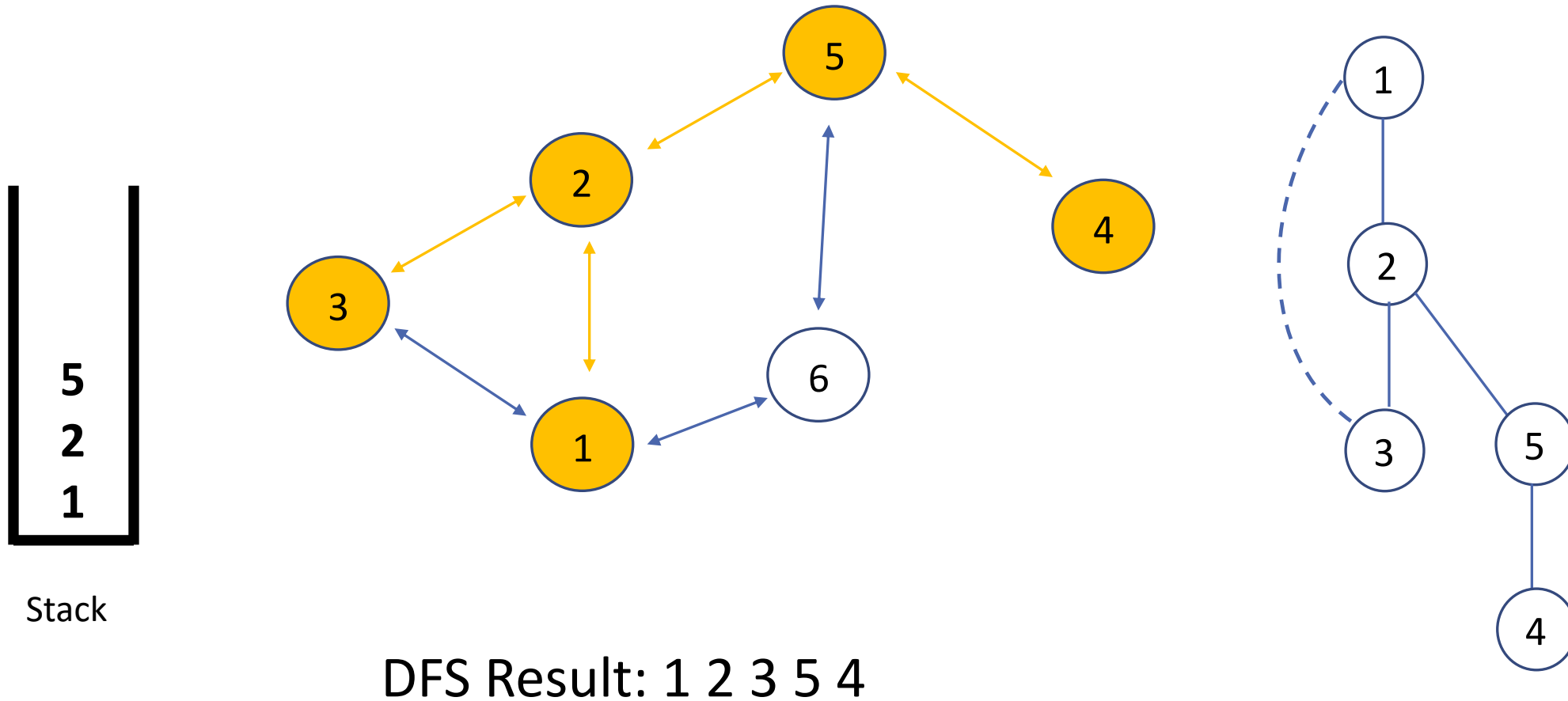


DFS Result: 1 2 3 5

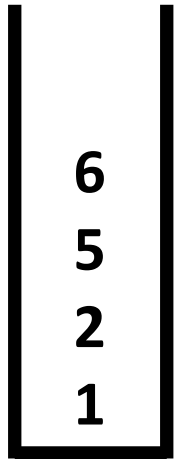
Example 1: Depth-First Search (DFS)



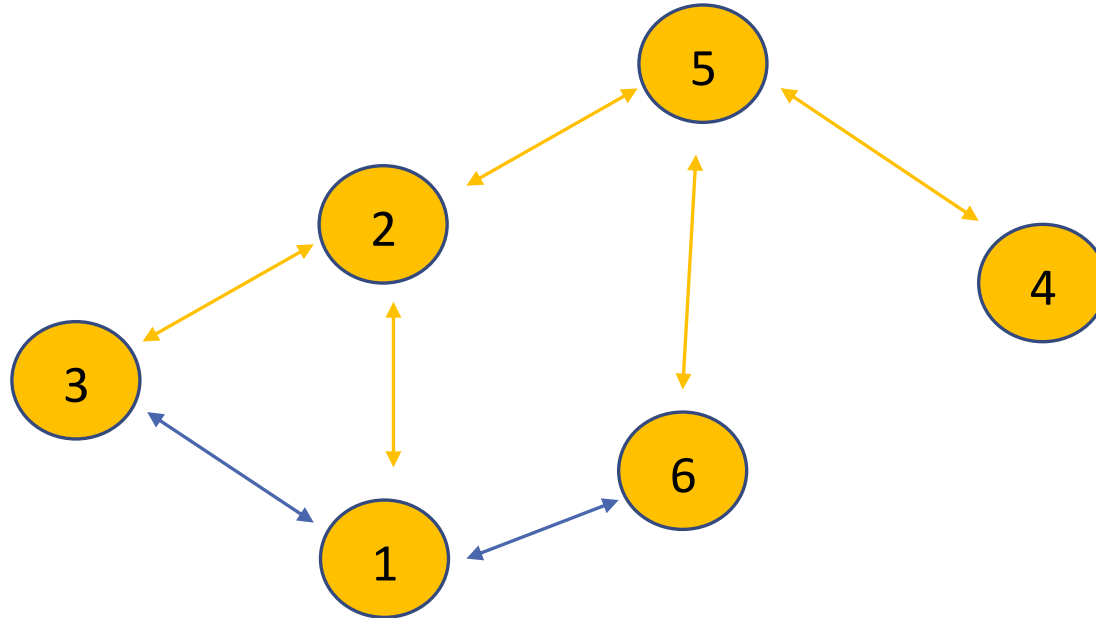
Example 1: Depth-First Search (DFS)



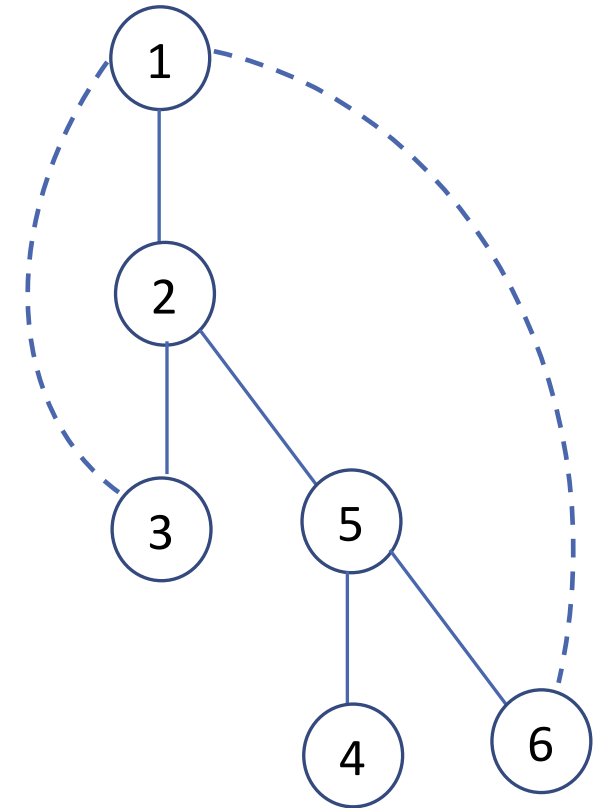
Example 1: Depth-First Search (DFS)



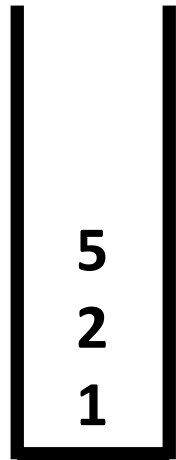
Stack



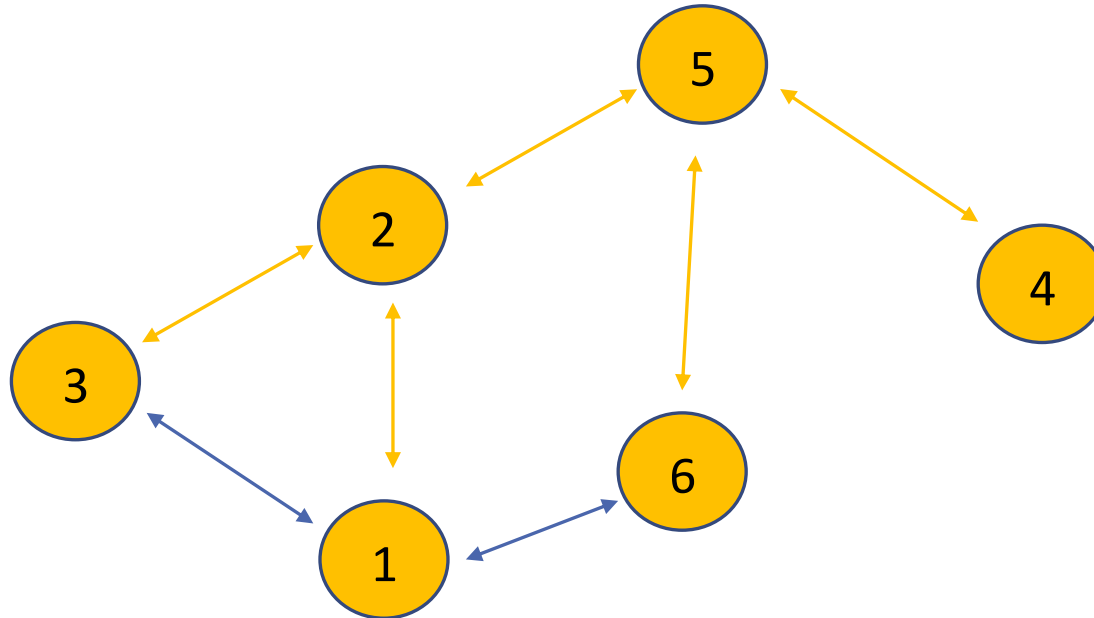
DFS Result: 1 2 3 5 4 6



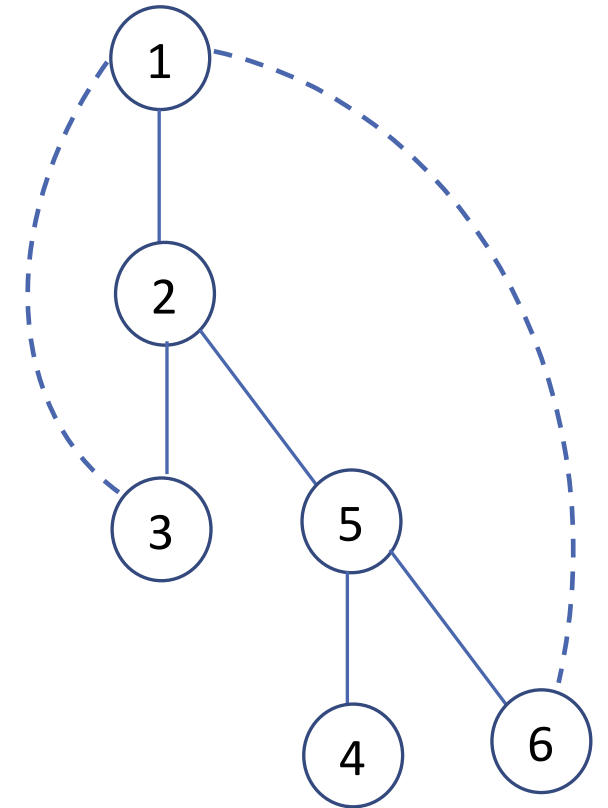
Example 1: Depth-First Search (DFS)



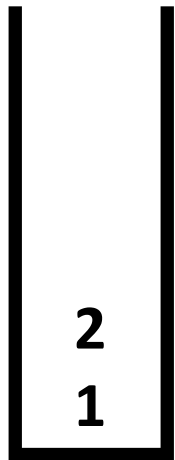
Stack



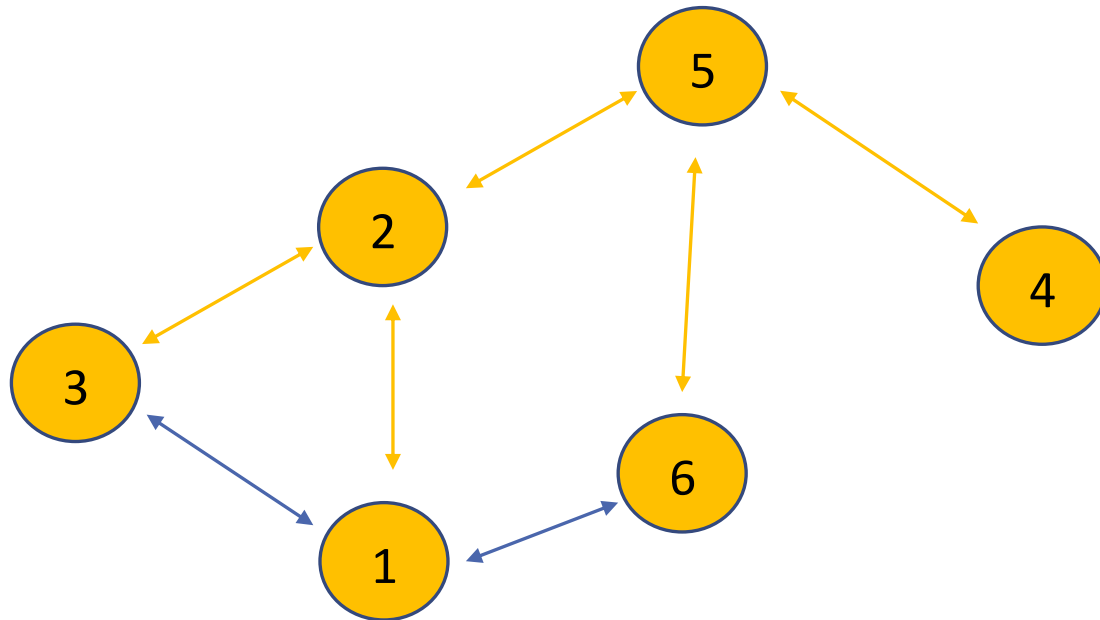
DFS Result: 1 2 3 5 4 6



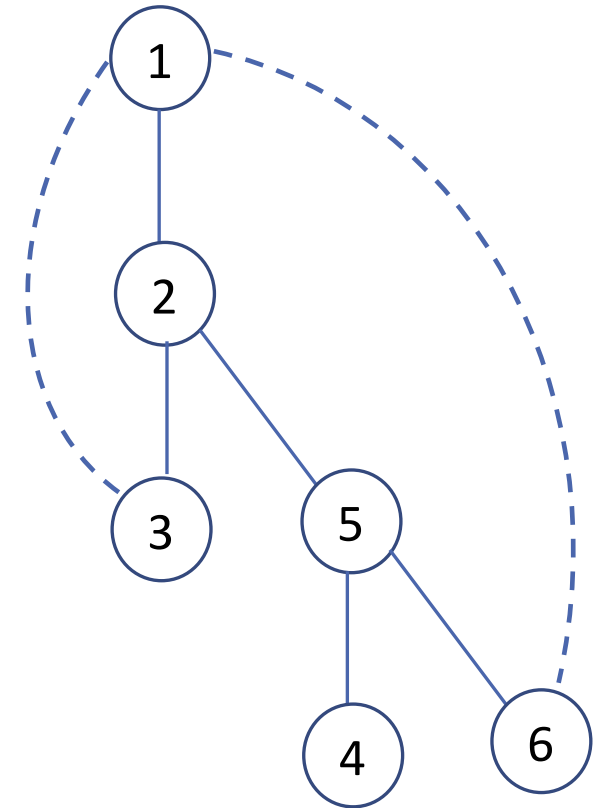
Example 1: Depth-First Search (DFS)



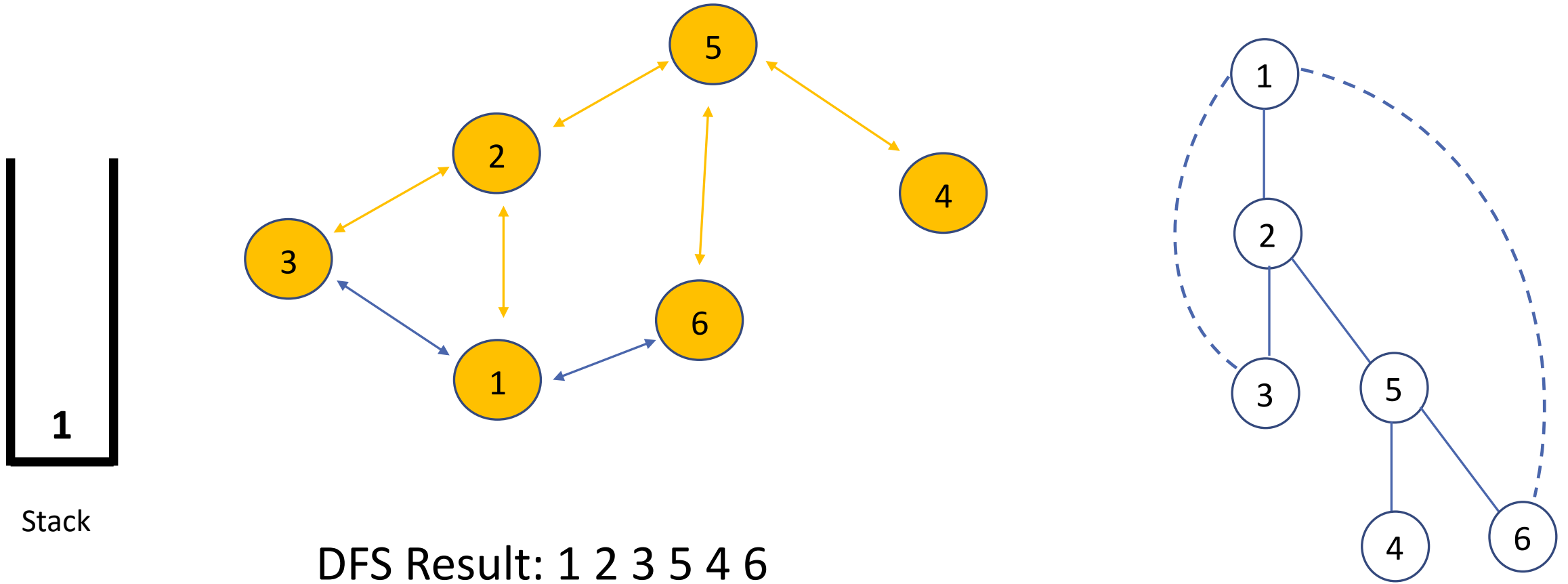
Stack



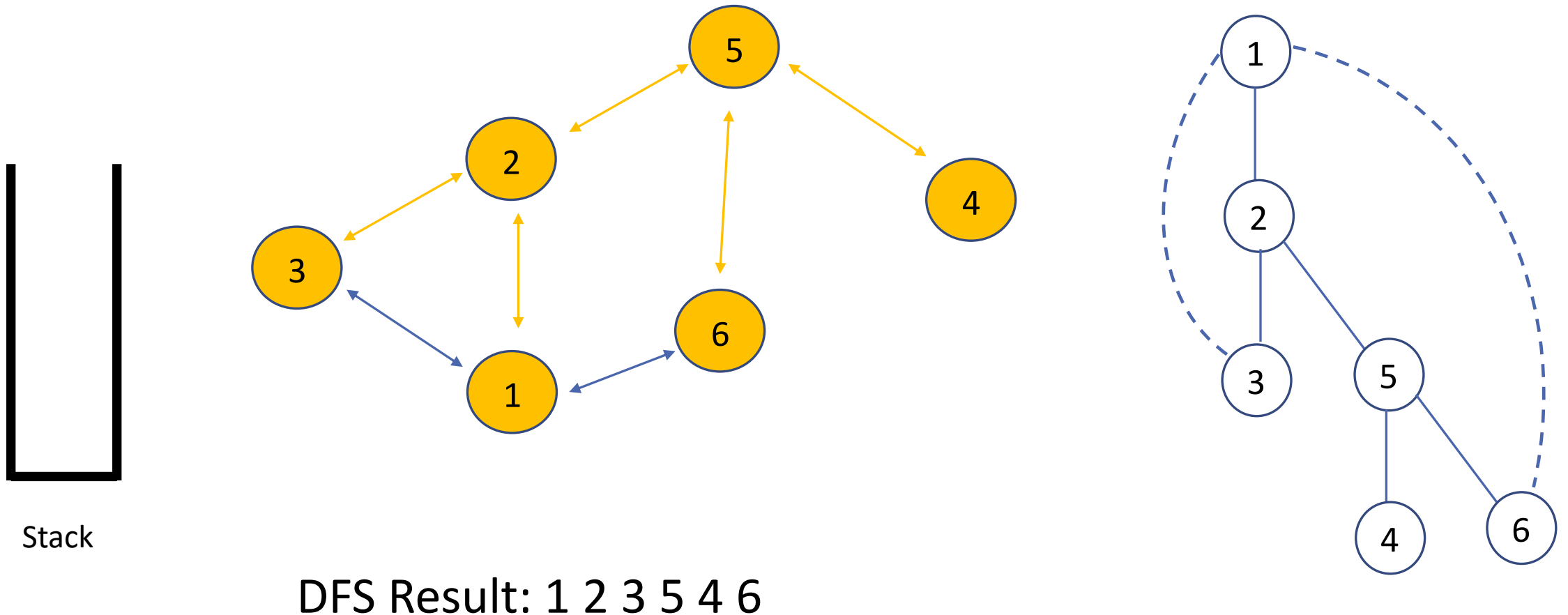
DFS Result: 1 2 3 5 4 6



Example 1: Depth-First Search (DFS)



Example 1: Depth-First Search (DFS)



Pseudocode of DFS

ALGORITHM $DFS(G)$

```
//Implements a depth-first search traversal of a given graph
//Input: Graph  $G = \langle V, E \rangle$ 
//Output: Graph  $G$  with its vertices marked with consecutive integers
//          in the order they are first encountered by the DFS traversal
mark each vertex in  $V$  with 0 as a mark of being “unvisited”
count  $\leftarrow 0$ 
for each vertex  $v$  in  $V$  do
    if  $v$  is marked with 0
        dfs( $v$ )

dfs( $v$ )
//visits recursively all the unvisited vertices connected to vertex  $v$ 
//by a path and numbers them in the order they are encountered
//via global variable count
count  $\leftarrow$  count + 1; mark  $v$  with count
for each vertex  $w$  in  $V$  adjacent to  $v$  do
    if  $w$  is marked with 0
        dfs( $w$ )
```


Depth-First Search (DFS)

- The **brevity** of the DFS pseudocode and the ease with which it can be performed by hand may create a wrong impression about the **level of sophistication** of this algorithm.
- How efficient is depth-first search?
- It is not difficult to see that this algorithm is, in fact, quite efficient since it takes just the **time proportional to the size of the data structure** used for representing the graph in question.
- Thus, for the **adjacency matrix representation**, the traversal time is in $\Theta(|V|^2)$,
- For the **adjacency list representation**, it is in $\Theta(|V| + |E|)$.

Depth-First Search (DFS)

- A DFS **forest** is obtained as a by-product of a DFS traversal
- We can look at it as the given graph with its edges classified by the DFS traversal into two disjoint classes: tree edges and back edges.
- **Tree edges** are edges used by the DFS traversal to reach previously unvisited vertices.
- If we consider only the edges in this class, we will indeed get a forest.
- **Back edges** connect vertices to previously visited vertices other than their immediate predecessors in the traversal.

Depth-First Search (DFS)

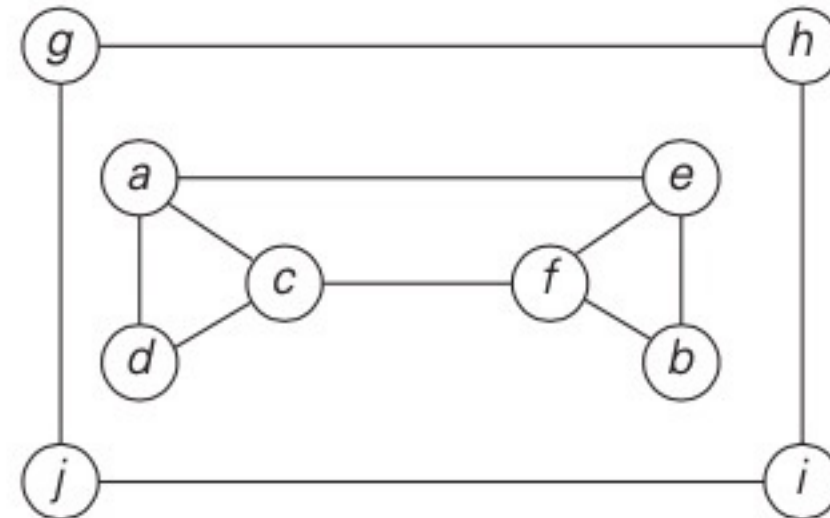
- A DFS traversal itself and the forest-like representation of the graph it provides have proved to be **extremely helpful for the development of efficient algorithms** for checking many important properties of graphs.
- The DFS yields **two orderings** of vertices: the order in which the vertices are **reached for the first time** (pushed onto the stack) and the order in which the vertices **become dead ends** (popped off the stack).
- These orders are qualitatively different, and various applications can take advantage of either of them.

Depth-First Search (DFS)

- Important elementary applications of DFS include **checking connectivity and checking acyclicity** of a graph.
- Since DFS halts after visiting all the vertices connected by a path to the starting vertex, checking a graph's connectivity can be done as follows.
- Start a DFS traversal at an arbitrary vertex and check, after the algorithm halts, whether **all the vertices of the graph will have been visited**.
- If they have, the graph is **connected**; otherwise, it is **not connected**.
- More generally, we can use DFS for **identifying connected components** of a graph.

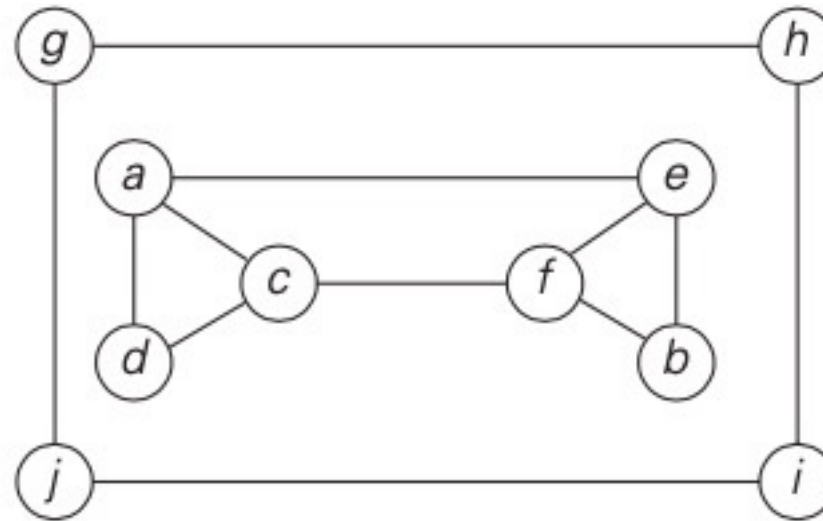
Depth-First Search (DFS)

- As for checking for a cycle presence in a graph, we can take advantage of the graph's representation in the form of a DFS forest.
- If the latter does **not have back edges**, the graph is clearly **acyclic**.
- If there is a back edge from some vertex u to its ancestor v (e.g., the back edge from d to a), the graph has a cycle that comprises the path from v to u via a sequence of tree edges in the DFS forest followed by the back edge from u to v .



Example 2: Depth-First Search (DFS)

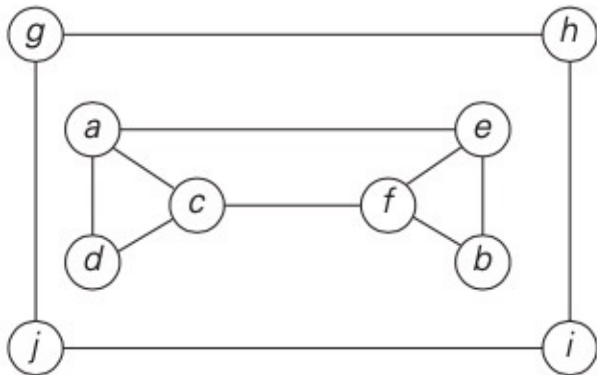
- Starting at vertex “a” and resolving ties by the vertex alphabetical order, traverse the graph by depth-first search and write depth-first search list and illustrate DFS tree.



Example 2: Depth-First Search (DFS)

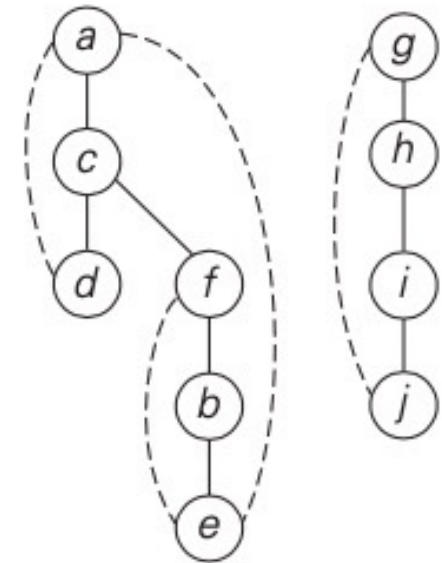
Traversal's stack (the first subscript number indicates the order in which a vertex is visited, i.e., pushed onto the stack; the second one indicates the order in which it becomes a dead-end, i.e., popped off the stack).

Graph



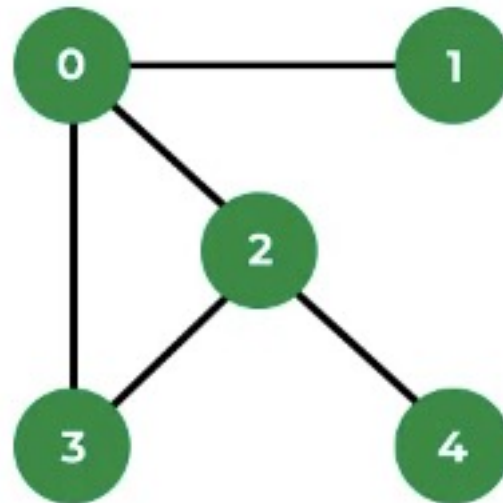
$d_{3,1}$	$e_{6,2}$	$j_{10,7}$
$c_{2,5}$	$b_{5,3}$	$i_{9,8}$
$a_{1,6}$	$f_{4,4}$	$h_{8,9}$
		$g_{7,10}$

DFS forest with the tree and back edges shown with solid and dashed lines, respectively.



Example 3: Depth-First Search (DFS)

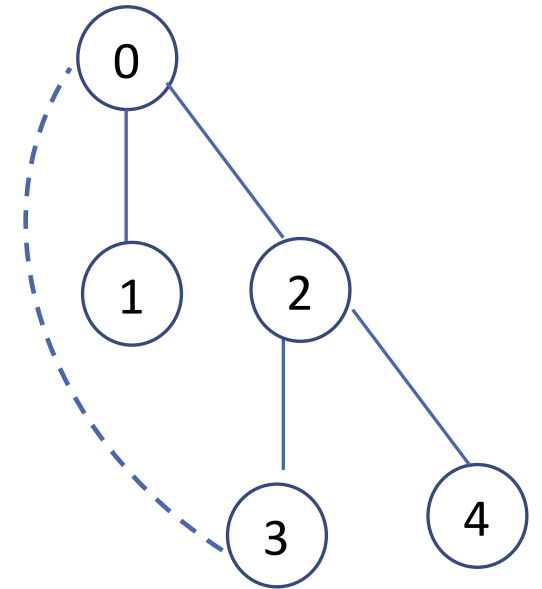
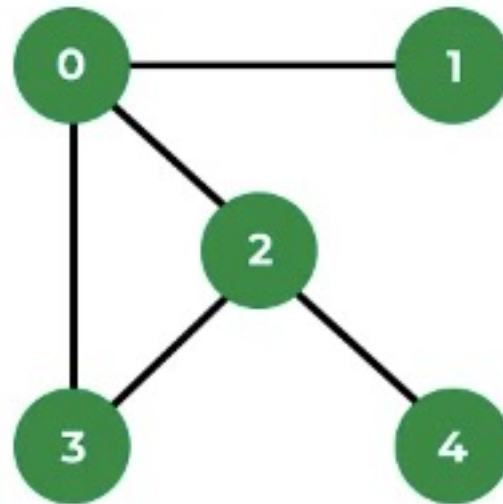
- Starting at vertex "0" and resolving ties by the vertex alphabetical order, traverse the graph by depth-first search and write down depth-first search list.



Example 3: Depth-First Search (DFS)

○Solution:

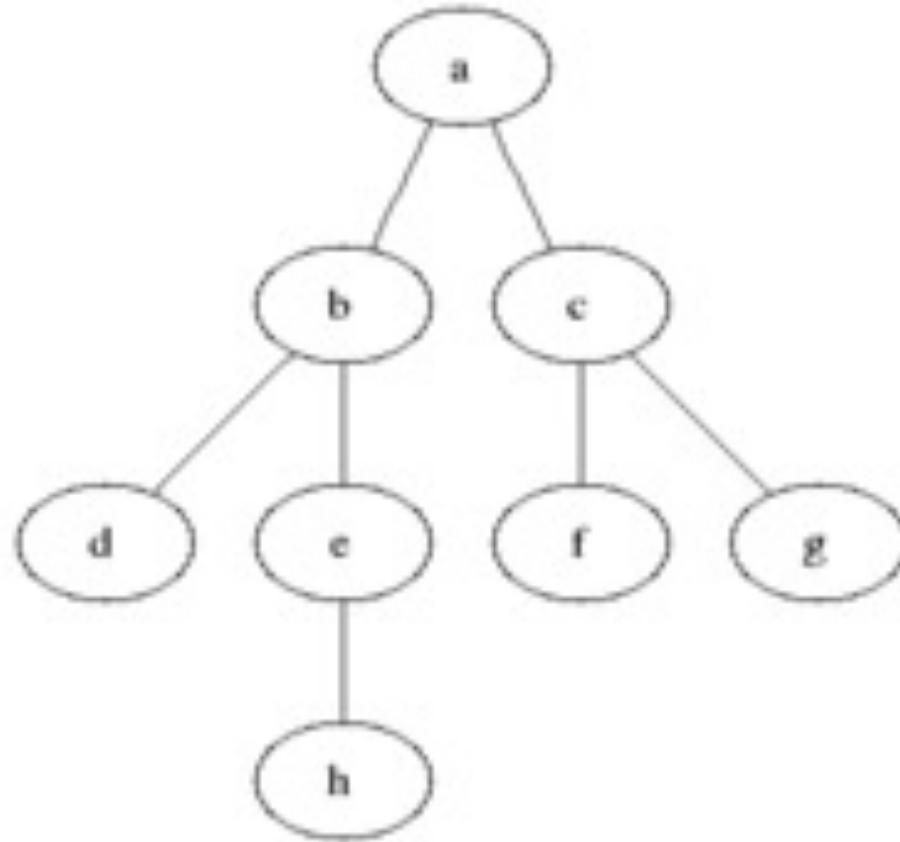
DFS Result: 0 1 2 3 4



Main Points of DFS

- DFS can be implemented with graphs represented as:
 - adjacency matrices: $\Theta(|V|^2)$
 - adjacency lists: $\Theta(|V| + |E|)$
- Yields two distinct ordering of vertices:
 - Order in which vertices are first encountered (pushed onto stack)
 - Order in which vertices become dead-ends (popped off stack)
- Applications:
 - Checking connectivity, finding connected components
 - Checking acyclicity
 - Finding articulation points and biconnected components
 - Searching state-space of problems for solution (AI)

Breadth-first search (BFS)

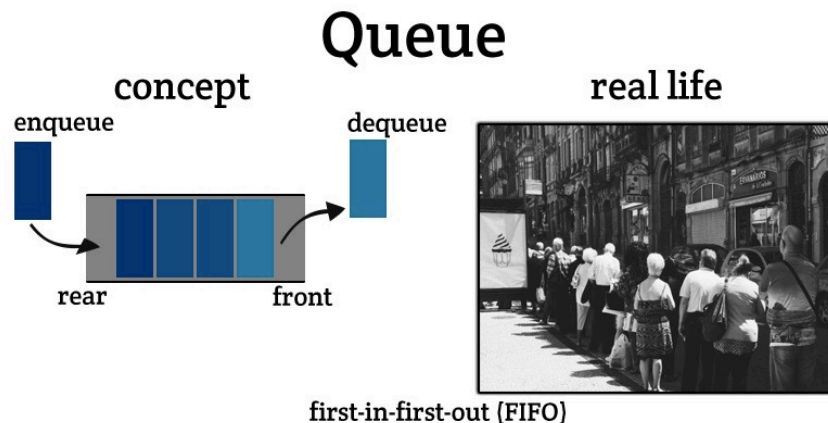


Breadth-first search (BFS)

- Breadth-first search is a traversal for the **cautious**.
- It proceeds in a **concentric** manner by visiting first all the vertices that are adjacent to a starting vertex, then all unvisited vertices two edges apart from it, and so on, until all the vertices in the same connected component as the starting vertex are visited.
- If there still remain unvisited vertices, the algorithm has to be **restarted at an arbitrary vertex** of another connected component of the graph.

Breadth-first search (BFS)

- It is convenient to use a **queue** to trace the operation of breadth-first search.
- The queue is initialized with the traversal's starting vertex, which is marked as visited.
- On each iteration, the algorithm identifies all unvisited vertices that are adjacent to the front vertex, marks them as visited, and adds them to the queue; after that, the front vertex is removed from the queue.
- Visits graph vertices by moving across to all the neighbors of last visited vertex

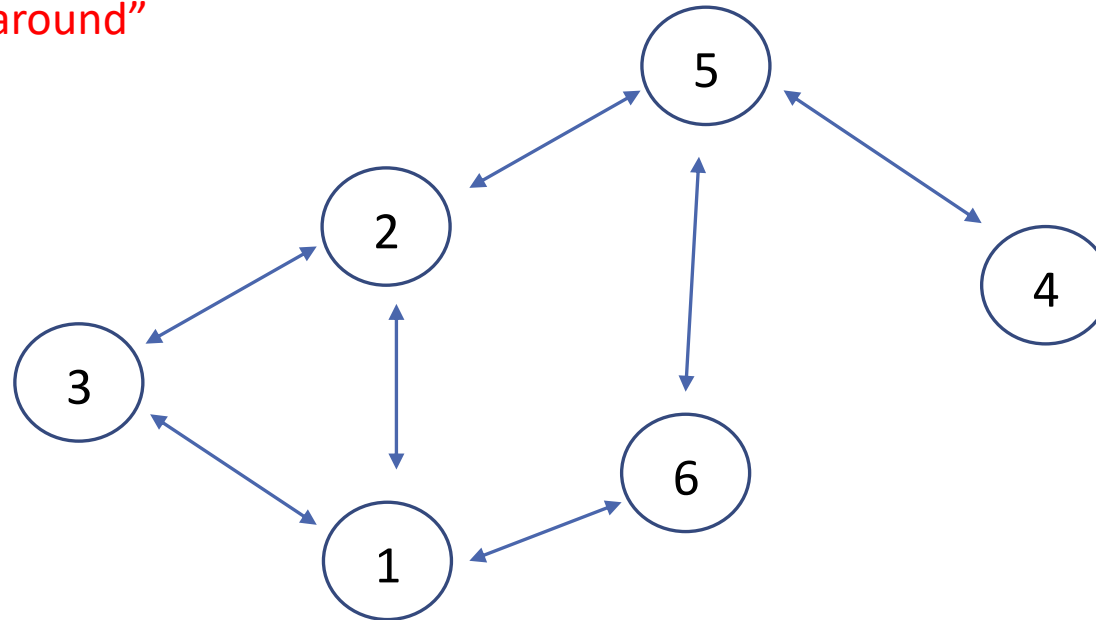


Example 1: Breadth-first search (BFS)

- The essence of BFS: “Explore around”



Queue

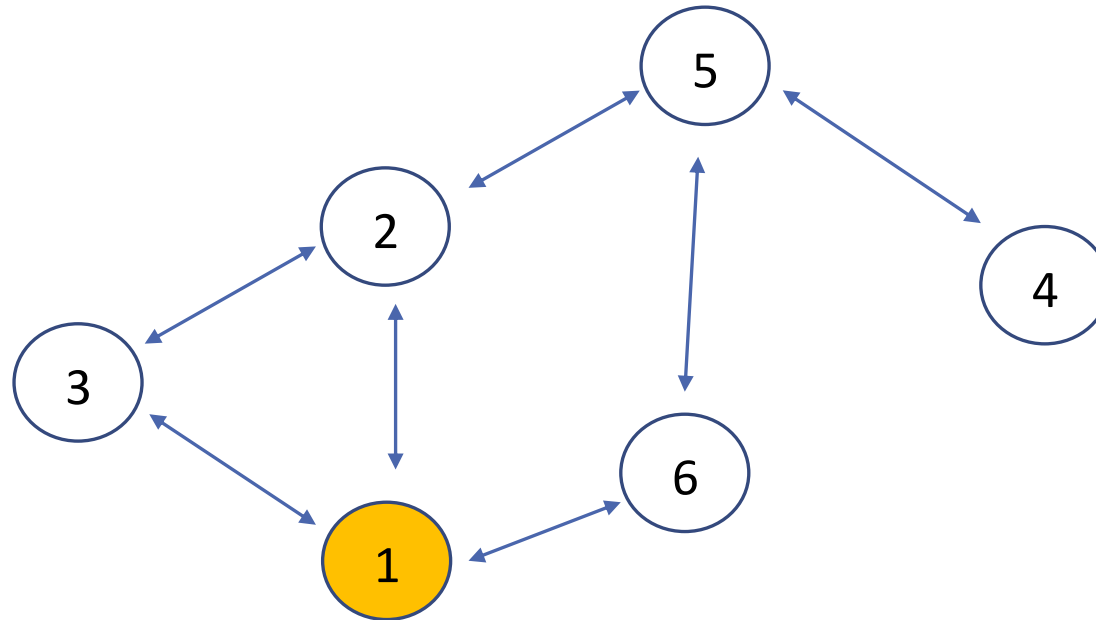


BFS Result:

Example 1: Breadth-first search (BFS)

1

Queue

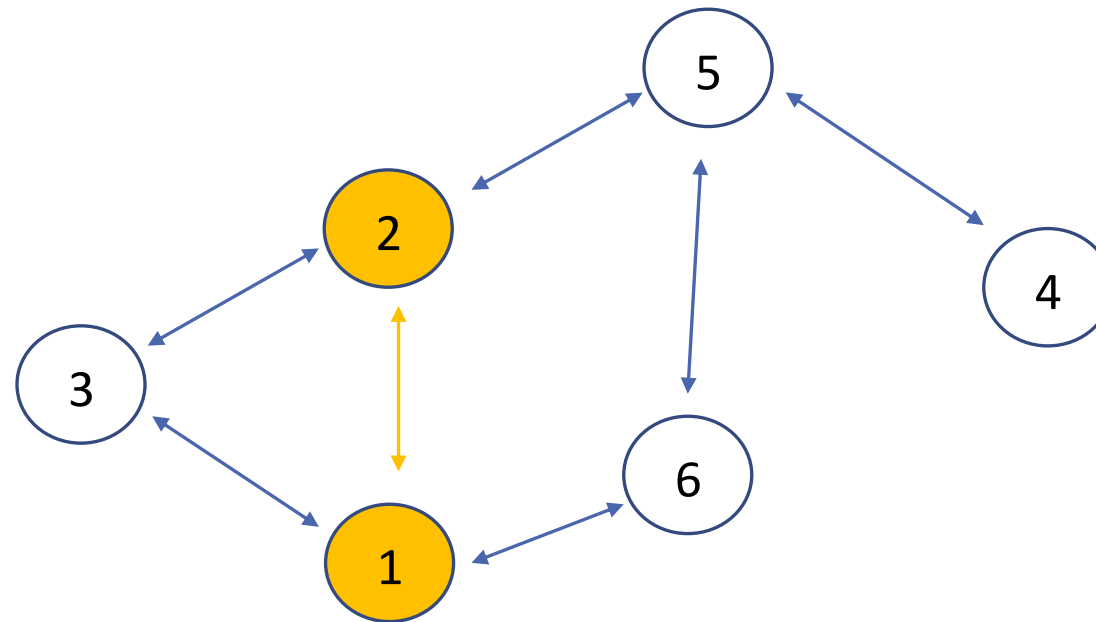


BFS Result: 1

Example 1: Breadth-first search (BFS)

2 1

Queue

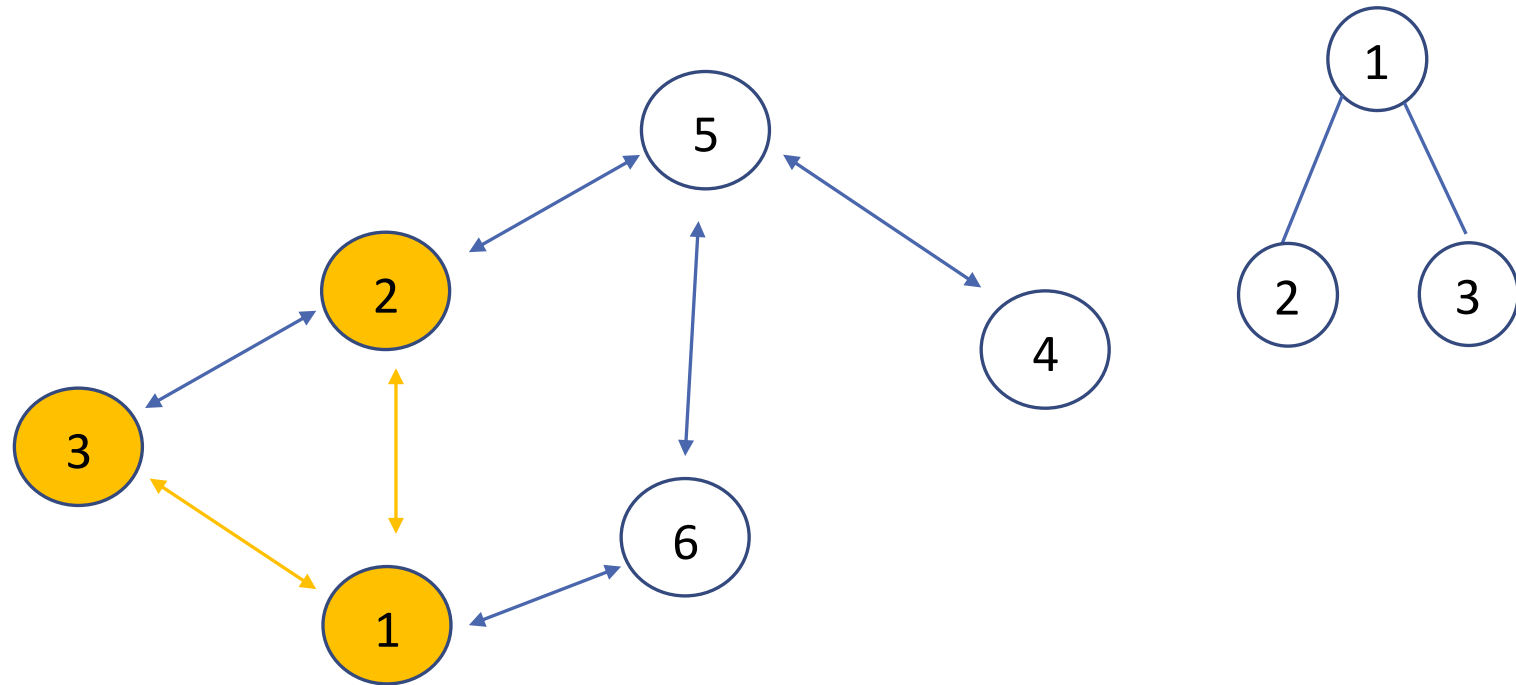


BFS Result: 1 2

Example 1: Breadth-first search (BFS)

3 2 1

Queue

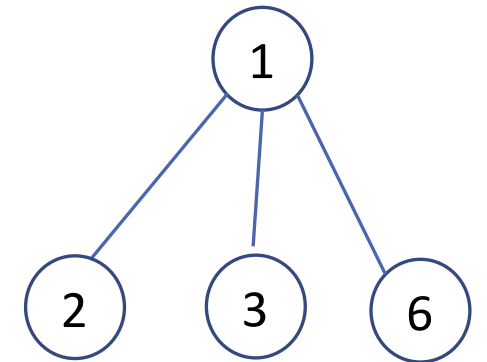
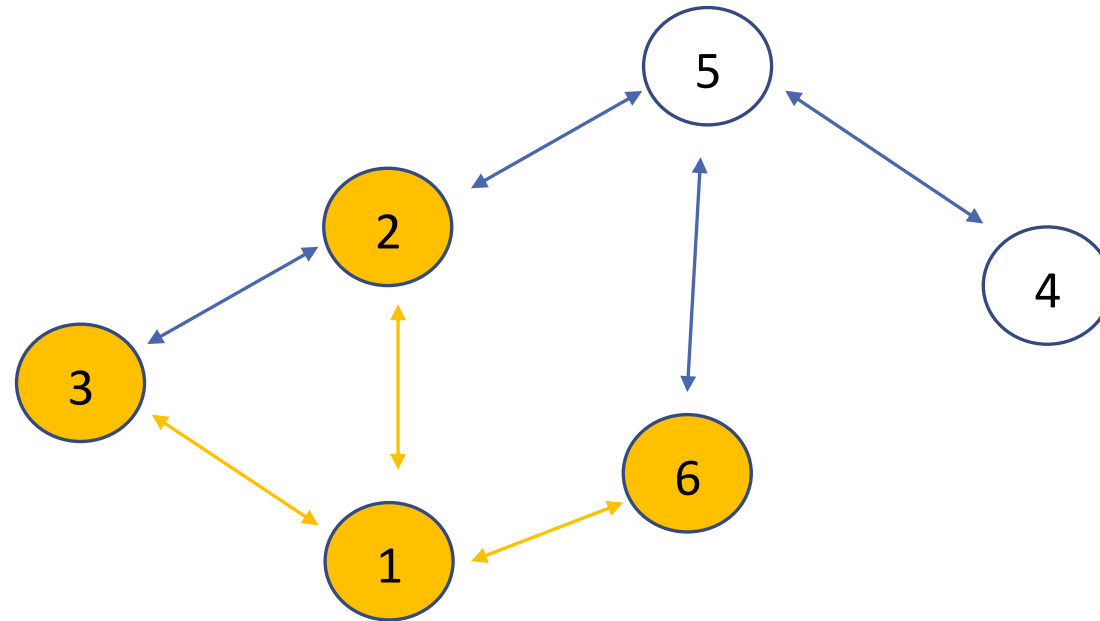


BFS Result: 1 2 3

Example 1: Breadth-first search (BFS)

6 3 2 1

Queue

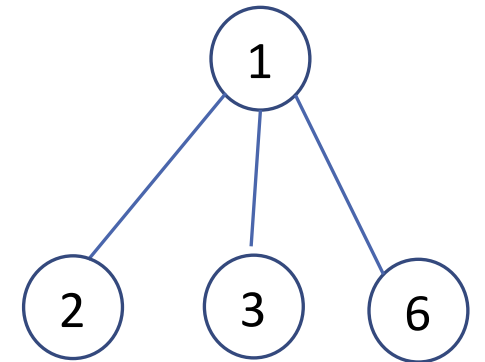
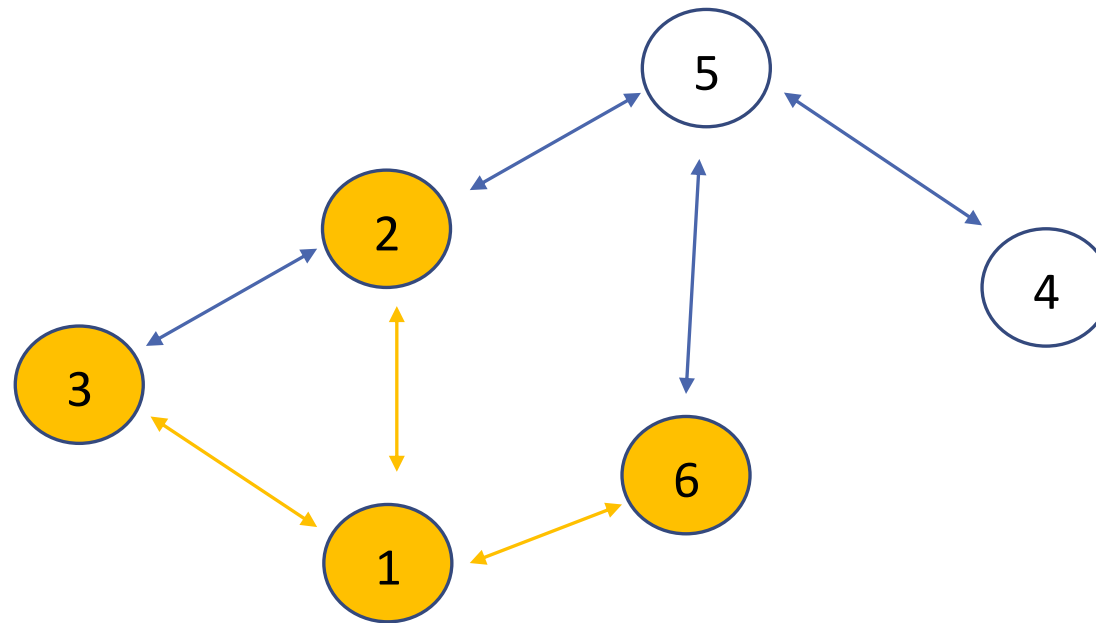


BFS Result: 1 2 3 6

Example 1: Breadth-first search (BFS)

6 3 2

Queue

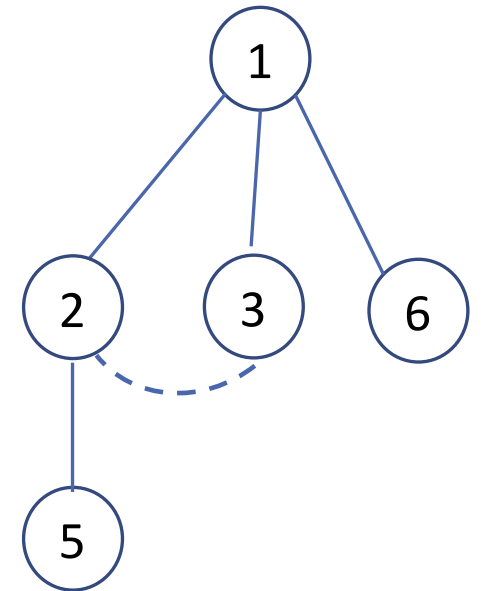
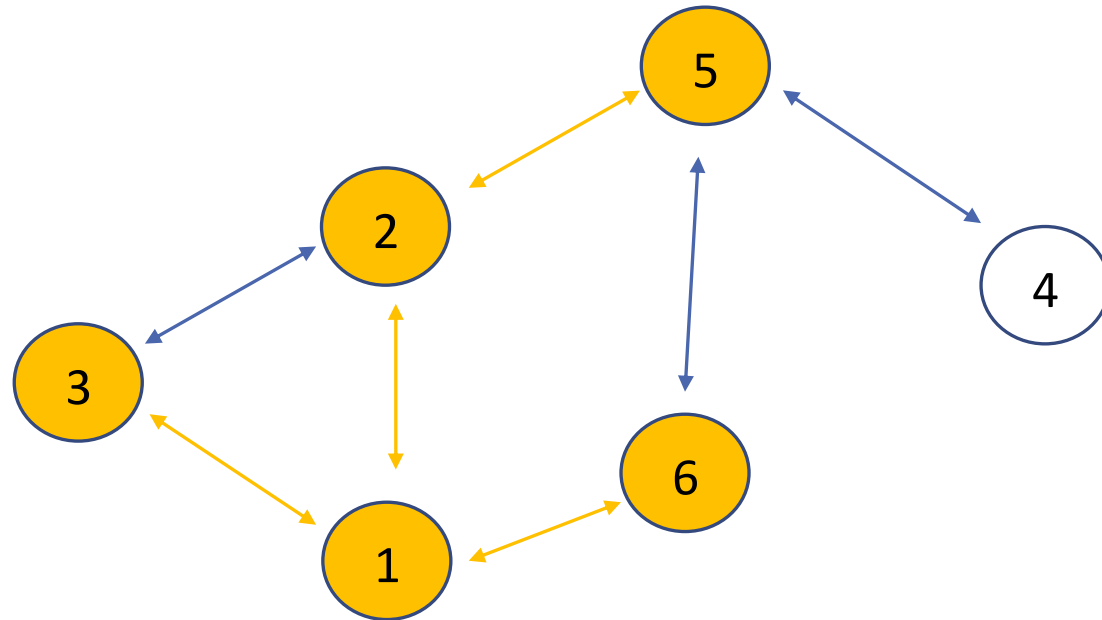


BFS Result: 1 2 3 6

Example 1: Breadth-first search (BFS)

5 6 3 2

Queue

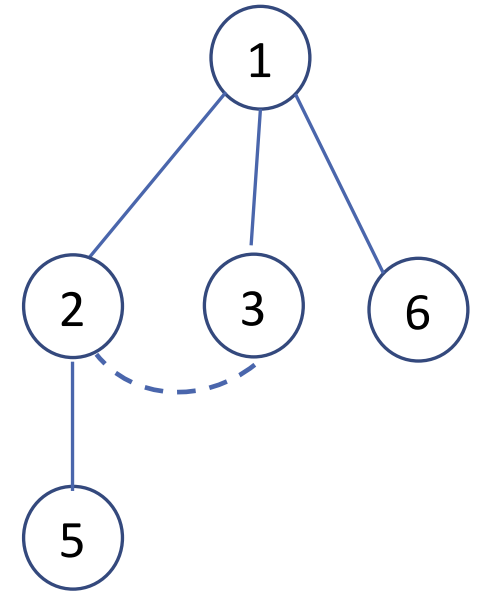
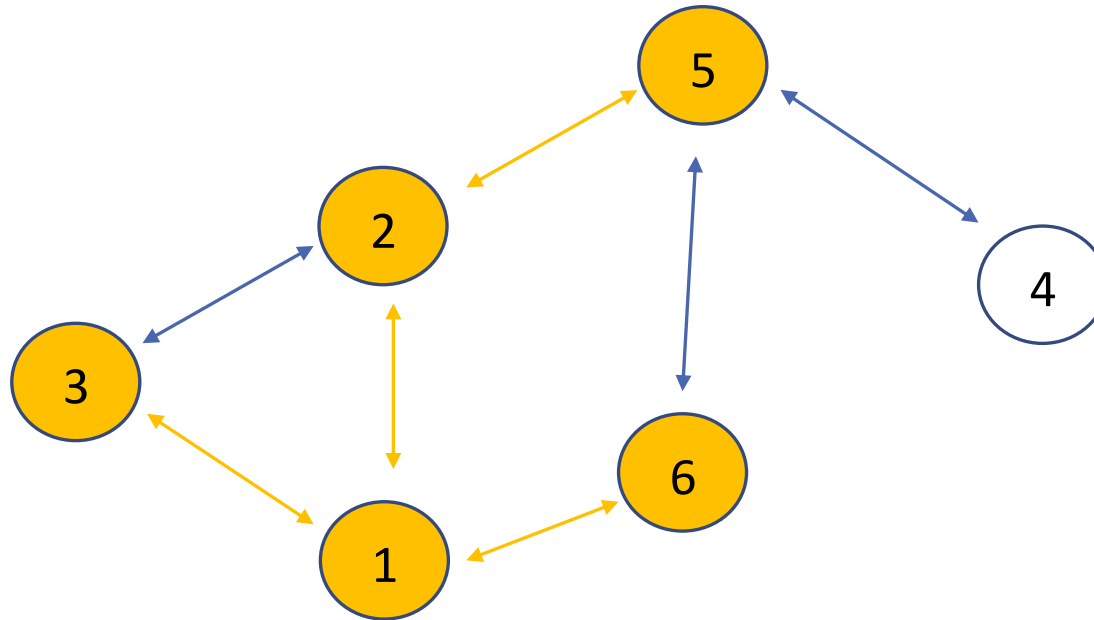


BFS Result: 1 2 3 6 5

Example 1: Breadth-first search (BFS)

5 6 3

Queue

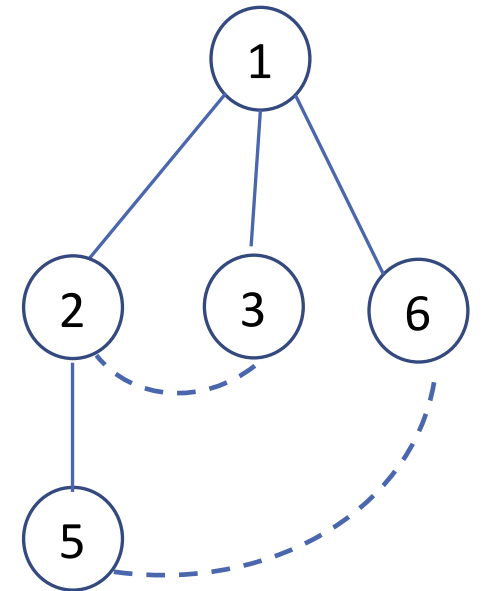
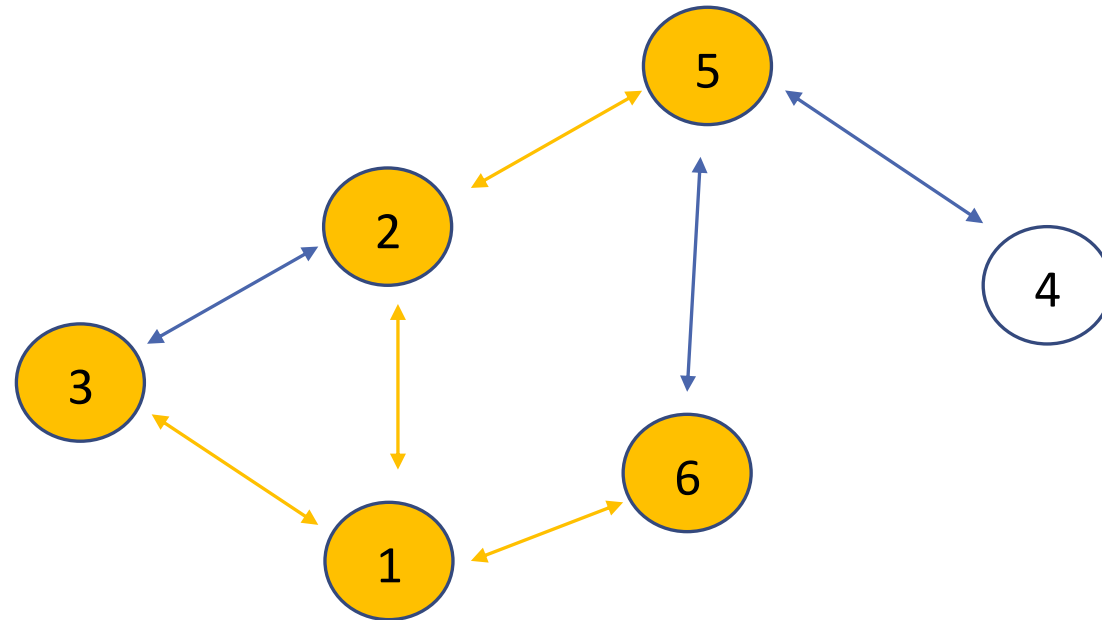


BFS Result: 1 2 3 6 5

Example 1: Breadth-first search (BFS)

5 6

Queue

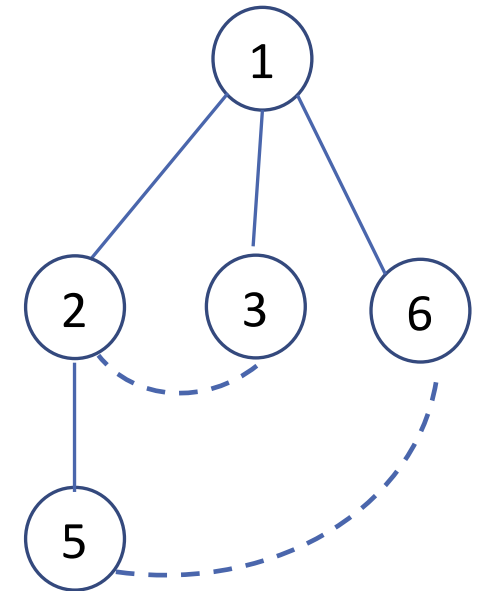
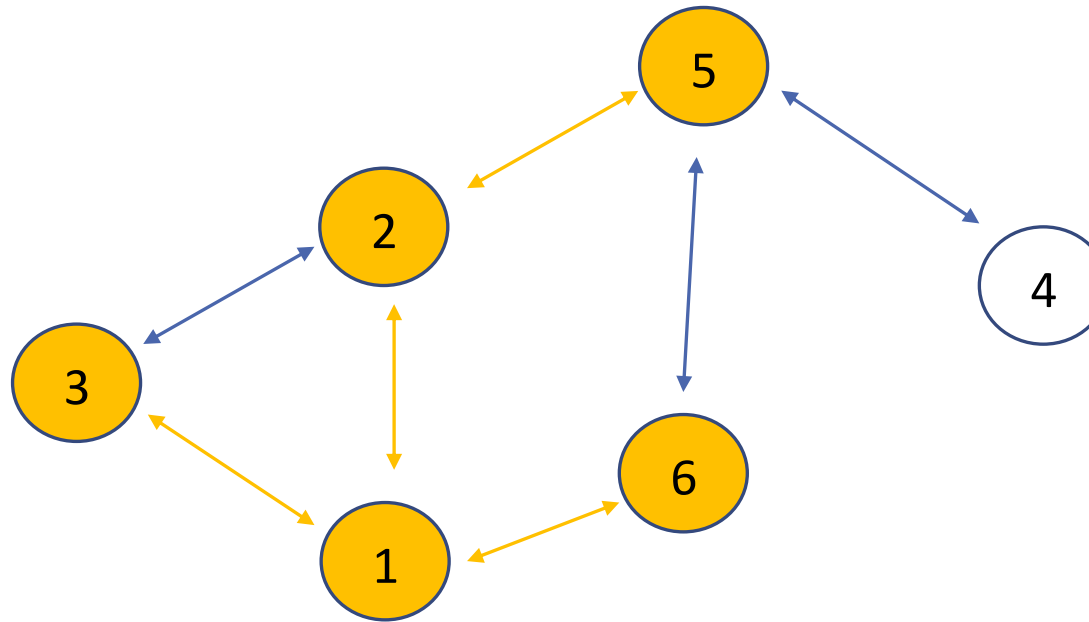


BFS Result: 1 2 3 6 5

Example 1: Breadth-first search (BFS)

5

Queue

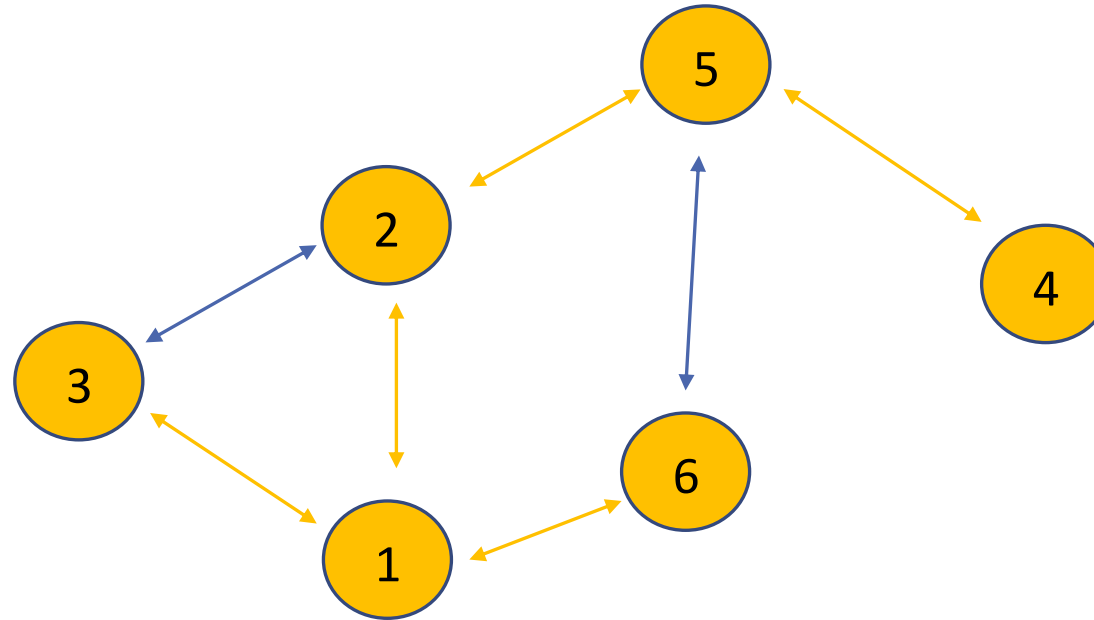


BFS Result: 1 2 3 6 5

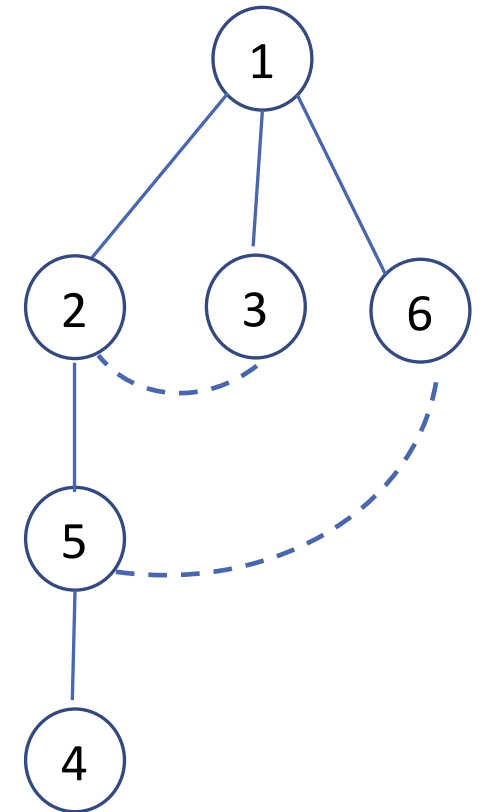
Example 1: Breadth-first search (BFS)

4 5

Queue



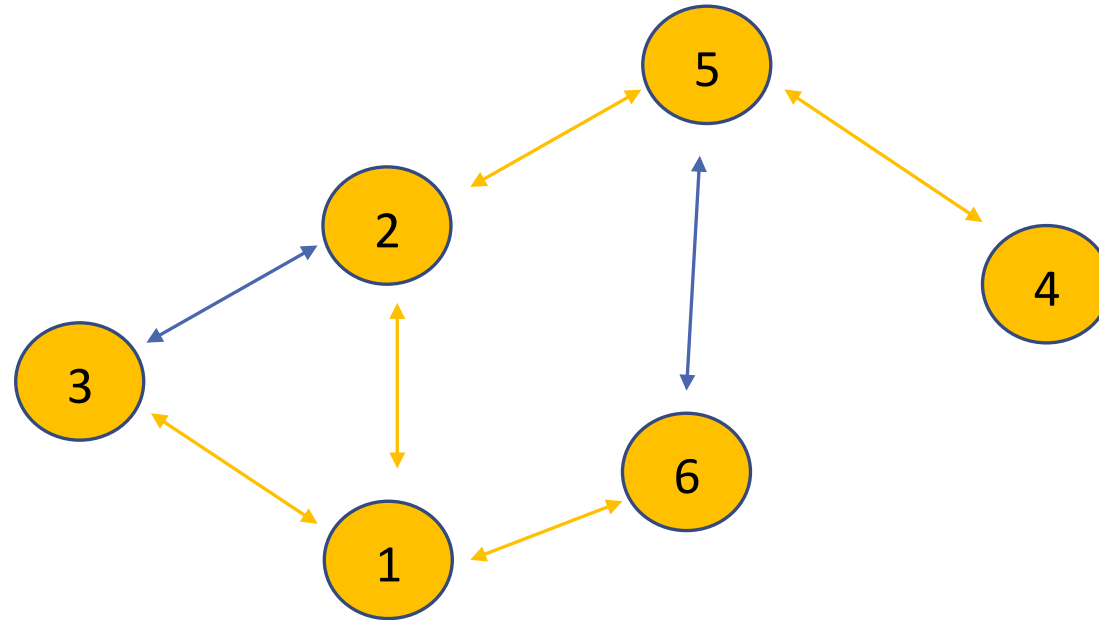
BFS Result: 1 2 3 6 5 4



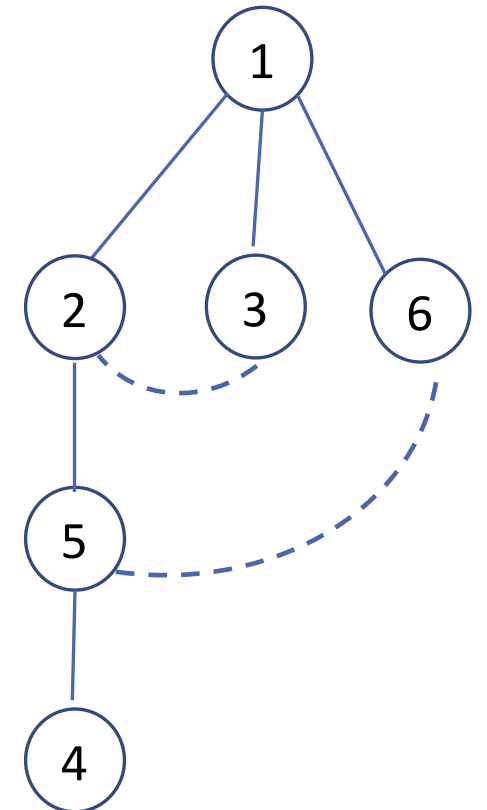
Example 1: Breadth-first search (BFS)

4

Queue

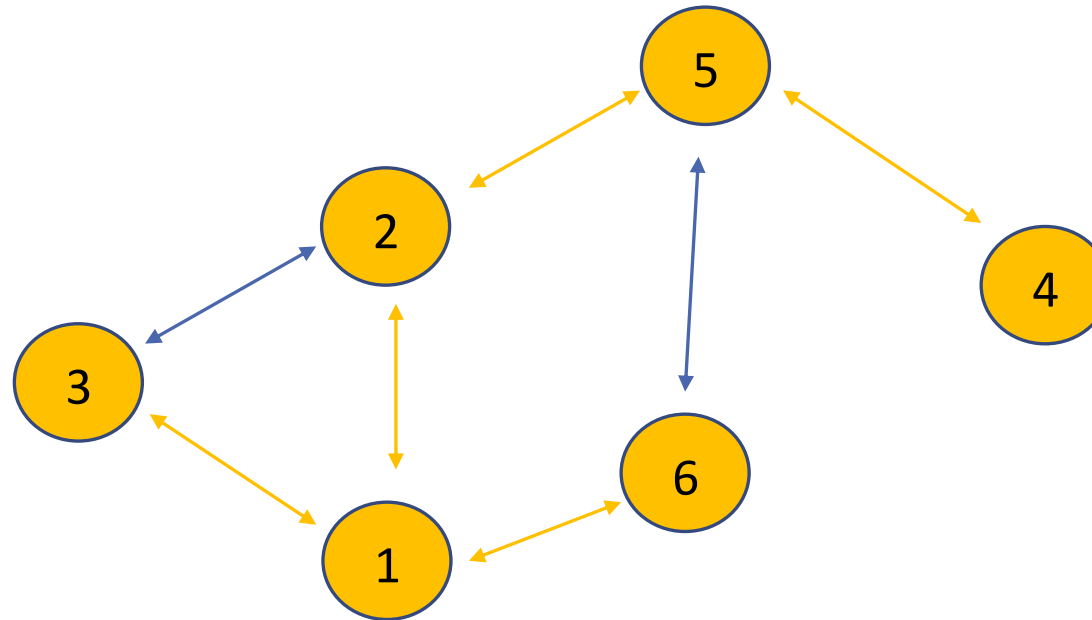


BFS Result: 1 2 3 6 5 4

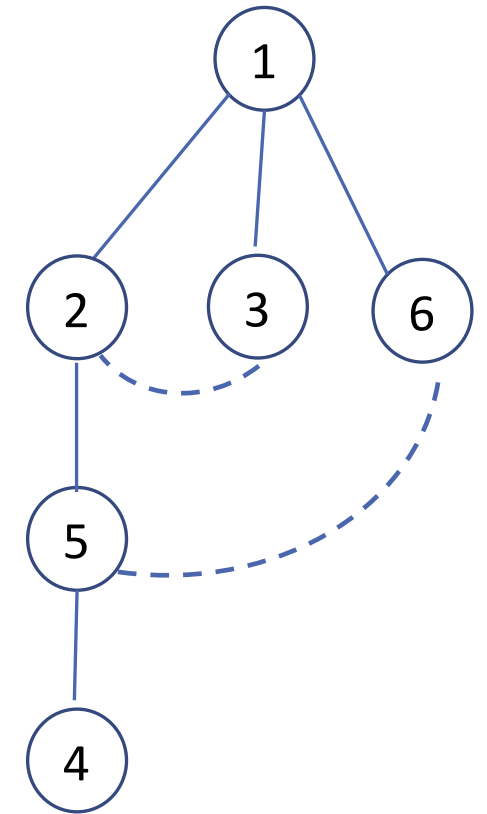


Example 1: Breadth-first search (BFS)

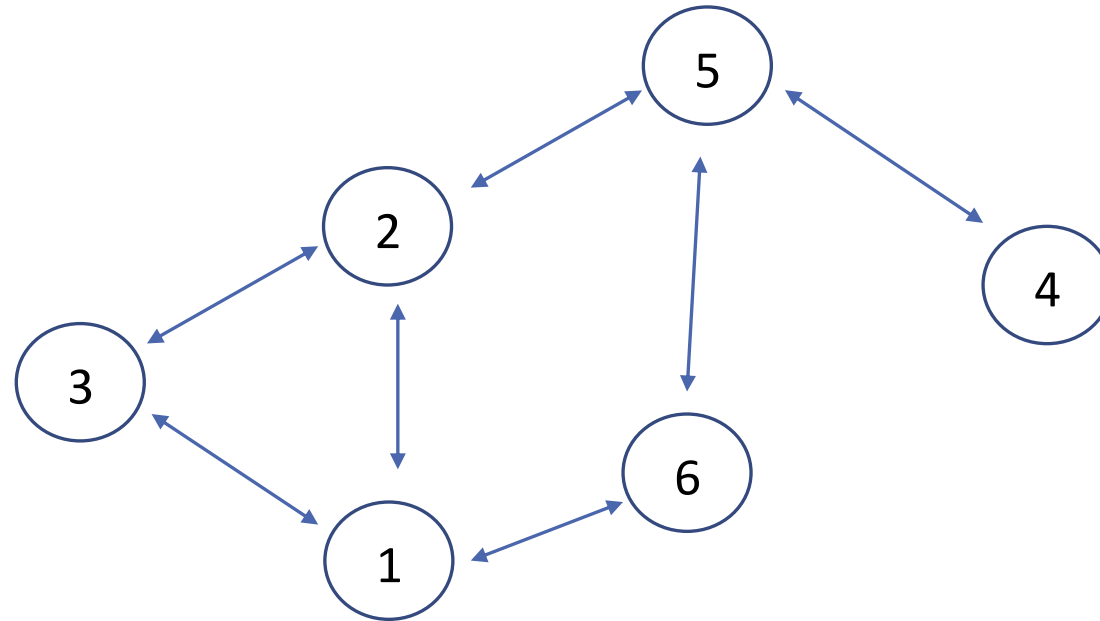
Queue



BFS Result: 1 2 3 6 5 4



DFS-BFS



DFS Result: 1 2 3 5 4 6

BFS Result: 1 2 3 6 5 4

Pseudocode of BFS

ALGORITHM $BFS(G)$

//Implements a breadth-first search traversal of a given graph

//Input: Graph $G = \langle V, E \rangle$

//Output: Graph G with its vertices marked with consecutive integers

// in the order they are visited by the BFS traversal
mark each vertex in V with 0 as a mark of being “unvisited”

$count \leftarrow 0$

for each vertex v in V **do**

if v is marked with 0

$bfs(v)$

$bfs(v)$

//visits all the unvisited vertices connected to vertex v

//by a path and numbers them in the order they are visited

//via global variable $count$

$count \leftarrow count + 1$; mark v with $count$ and initialize a queue with v

while the queue is not empty **do**

for each vertex w in V adjacent to the front vertex **do**

if w is marked with 0

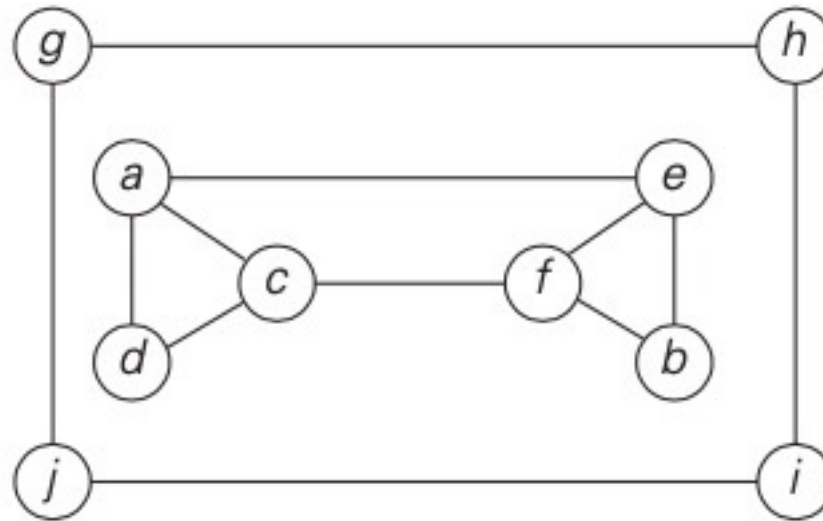
$count \leftarrow count + 1$; mark w with $count$

 add w to the queue

 remove the front vertex from the queue

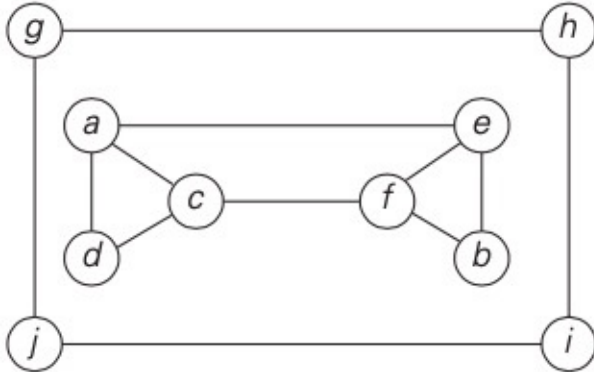
Example 2: BFS Traversal

- Starting at vertex “a” and resolving ties by the vertex alphabetical order, traverse the graph by breadth-first search and write breadth-first search list and illustrate BFS tree.



Example 2: BFS Traversal

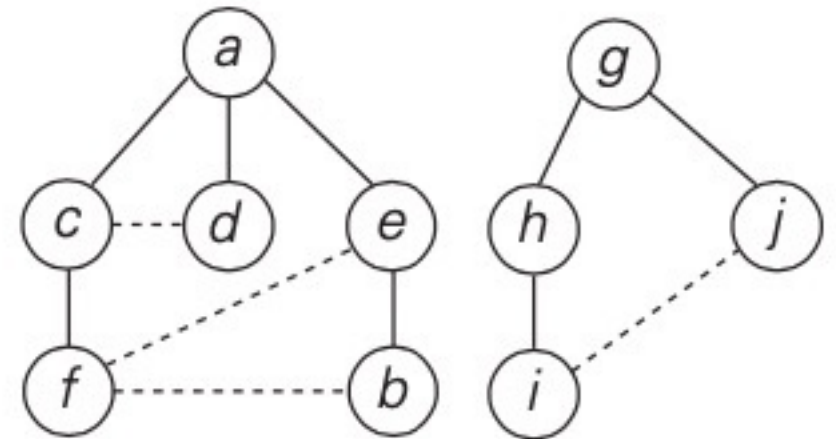
Graph



Traversal queue, with the numbers indicating **the order in which the vertices are visited**, i.e., added to (and removed from) the queue.

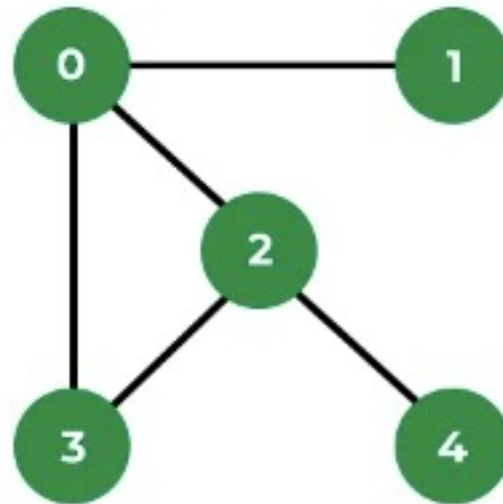
$a_1 c_2 d_3 e_4 f_5 b_6$
 $g_7 h_8 j_9 i_{10}$

BFS forest with the tree and cross edges shown with solid and dotted lines



Example 3: Breadth-first search (BFS)

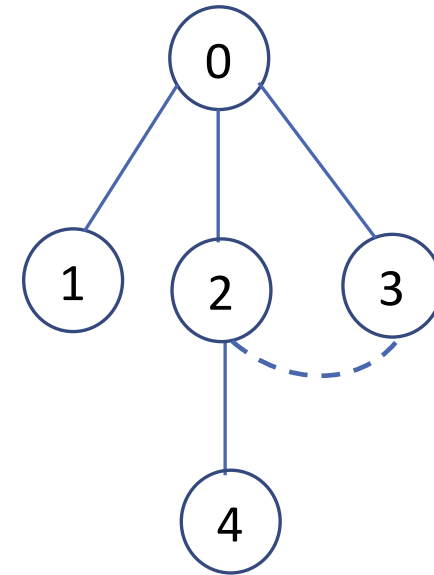
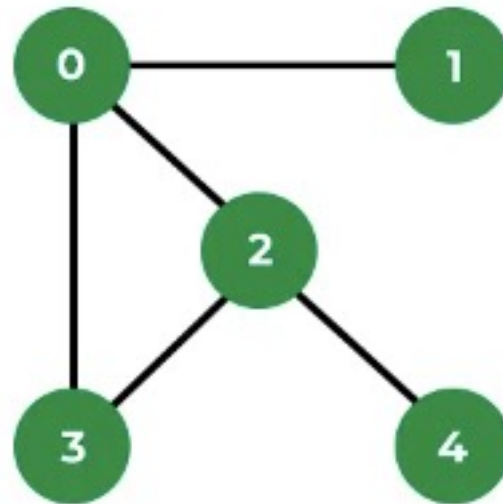
- Starting at vertex “0” and resolving ties by the vertex alphabetical order, traverse the graph by breadth-first search and write down breadth-first search list.



Example 3: Breadth-first search (BFS)

○Solution:

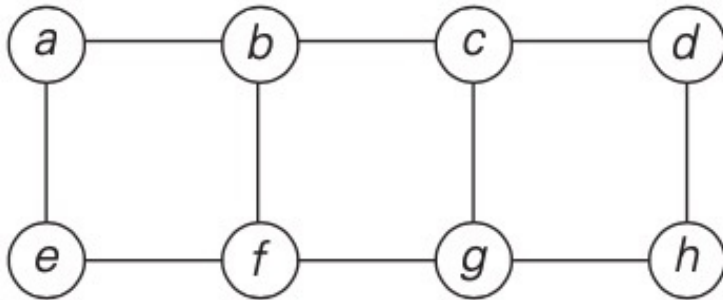
BFS Result: 0 1 2 3 4



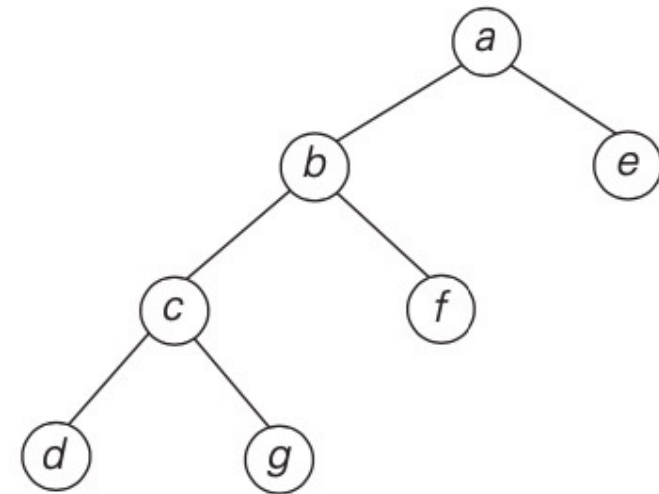
BFS Traversal

- Illustration of the BFS-based algorithm for finding a **minimum-edge path**.

Graph



Part of its BFS tree that identifies the minimum-edge path from a to g



Breadth-first search (BFS)

- Breadth-first search has the same efficiency as depth-first search.
- It is in $\Theta(|V|^2)$ for the adjacency matrix representation and in $\Theta(|V| + |E|)$ for the adjacency list representation.
- Unlike depth-first search, it yields a single ordering of vertices because the queue is a FIFO (first-in first-out) structure and hence the order in which vertices are added to the queue is the same order in which they are removed from it.

Breadth-first search (BFS)

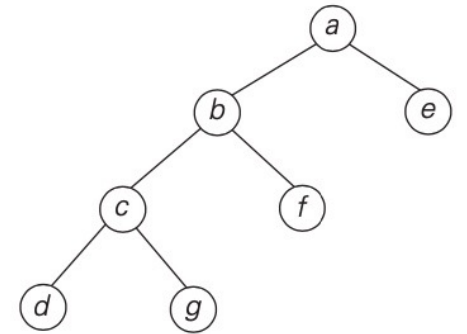
- As to the structure of a BFS forest of an undirected graph, it can also have two kinds of edges: **tree edges** and **cross edges**.
- Tree edges are the ones used to reach previously unvisited vertices.
- Cross edges connect vertices to those visited before

Breadth-first search (BFS)

- BFS can be used to **check connectivity and acyclicity of a graph**, essentially in the same manner as DFS can.
- It is not applicable, however, for several less straightforward applications such as finding **articulation points**.
- On the other hand, it can be helpful in some situations where DFS cannot.

Breadth-first search (BFS)

- For example, BFS can be used for finding a path with the **fewest number of edges between two given vertices**.
- To do this, we start a BFS traversal at one of the two vertices and stop it as soon as the other vertex is reached.
- The simple path from the root of the BFS tree to the second vertex is the path sought.
- For example, path $a - b - c - g$ in the graph has **the fewest number of edges** among all the paths between vertices a and g .



Main Points of BFS

- BFS has same efficiency as DFS and can be implemented with graphs represented as:
 - Adjacency matrices: $\Theta(|V|^2)$
 - Adjacency lists: $\Theta(|V| + |E|)$
- Yields **single ordering** of vertices (order added/deleted from queue is the same)
- Applications: Same as DFS, but can also find paths from a vertex to all other vertices with the smallest number of edges

DFS vs. BFS

- Main facts about depth-first search (DFS) and breadth-first search (BFS)

	DFS	BFS
Data structure	a stack	a queue
Number of vertex orderings	two orderings	one ordering
Edge types (undirected graphs)	tree and back edges	tree and cross edges
Applications	connectivity, acyclicity, articulation points	connectivity, acyclicity, minimum-edge paths
Efficiency for adjacency matrix	$\Theta(V ^2)$	$\Theta(V ^2)$
Efficiency for adjacency lists	$\Theta(V + E)$	$\Theta(V + E)$

Summary

- Graph Traversal
- Depth First Search (DFS)
- Breadth First Search (BFS)

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.